SC2006 - Software Engineering
2023/24 Semester 2

Final Documentation (SRS)

Lab Group: SCSE

Group: Trinity

Name	Matriculation Number
Agarwal Ananya	U2223411C
Huang Shuyang	U2210979A
Jason Lim Jiasheng	U2221526B
Kalidoss Madhumitha	U2222332A
Rathee Apurv Singh	U2120533F
Wong Wen Jie Scott	U2122366G

Table of Contents

Overview **4**
Purpose 4
Expected Users 4
Development **4**
Platform Compatibility 4
App Architecture 5
Functional Requirements **6**
1. Login 6
2. Register for new account 6
3. Main Menu 6
4. Recipe Section 7
5. Eat-Out Section 8
6. Delivery Section 9
7. Account Section 9
Non-Functional Requirements **10**
Data Dictionary **11**
Use Case Diagram **13**
Use Case Description **14**
Register and Login 14
Recipes 23
Eat-Out 38
Delivery 45
Manage Account 52
Class Diagram **56**
ECB Diagram **57**
Sequence Diagram **58**
Dialog Map **67**
System Architecture and Design **68**
Design Practices **69**
Software Engineering Practices **69**
Testing **70**
Black Box Testing 70
1. Login 70
2. Register For A New Account 71
3. Recipe 73
4. Eat-Out 75
5. Delivery 76
White Box Testing (Basis Path Testing) 77
1.Registering for a new account 77
2. Login 79
3. Forgot Password 81
4. Searching for restaurants for delivery by their name or cuisine they serve 83

5. Searching for restaurants for delivery by their ratings 85
6. View navigation directions to eat-out restaurant 87
7. Saving a user-uploaded recipe 89
Appendix **90**

Overview

The advancement of mobile technology has revolutionised our approach to food services. Currently, however, consumers must navigate a cluttered landscape of various apps for services such as discovering new recipes and new restaurants.

This prompted the creation of Trinity, a comprehensive mobile application designed to consolidate essential food-related services into a single, streamlined platform. As a mobile application, Trinity is ideally suited for on-the-go access, allowing users to find recipes, restaurants, and delivery options wherever they are. The convenience of having all these services available in one app on a device that nearly everyone carries daily enhances user engagement and satisfaction. Additionally, the mobile format leverages built-in smartphone features such as GPS for location services. This seamless integration with mobile capabilities makes Trinity a highly accessible and efficient solution for all food-related needs.

Purpose

Trinity offers three primary services: providing a variety of recipes for home cooking, helping users discover local restaurants for dining out, and facilitating the search for eateries that deliver. This integration ensures that whether users wish to cook, eat out, or order in, Trinity supports their dining needs efficiently in one place.

Expected Users

Trinity is crafted for anyone with a passion for food. Whether you are a cooking enthusiast in search of new recipes, someone who wants to share their culinary creations, or if you simply prefer the convenience of eating out or having food delivered, Trinity is your ultimate culinary partner.

Development

The app is built using Flutter for the frontend, a popular cross-platform framework that allows for building natively compiled applications for both Android and iOS. We used Dart as the primary programming language, which serves as the coding backbone of flutter and Firebase for backend services like authentication and data storage.

Platform Compatibility

The app is designed to work on both Android and iOS devices. Flutter's "write once, run anywhere" philosophy ensures consistent behaviour and user interface across different devices.

App Architecture

The app integrates several Firebase services to enable various functionalities. Below is an outline of the primary components and their interconnections:

Flutter Frontend: The user interface and interaction logic are built with Flutter. This layer is responsible for displaying content, handling user input, and managing navigation within the app. Our Flutter frontend is enhanced with key FlutterFlow packages to elevate both functionality and user experience. For example, the FlutterFlow Utilities package offers vital functions for state management, data formatting, and other routine tasks. FlutterFlow Widgets contribute a range of pre-built UI elements like custom buttons and input fields, ensuring a consistent and functional interface. The FlutterFlow Theme package provides a unified look and feel across the app, standardising colours, fonts, and styles. Additionally, FlutterFlow Google Maps supports location tracking and map-based searches, crucial for the Eat-Out section of our application. These integrations streamline user interactions and improve the overall usability of the app.

Firebase Authentication: This service manages user authentication, allowing users to create accounts, log in, and reset passwords. Integration with Firebase Auth is straightforward in Flutter, providing a secure and scalable authentication solution.

Firebase Firestore: Firestore is used as a cloud-based NoSQL database to store and retrieve user-generated recipes, as well as other dynamic content. This database facilitates quick read and write operations, enabling real-time updates to users.

API Integrations:

1. **Recipe API**: The app makes API calls to fetch a list of recipes from an external source. This functionality allows users to browse and select from a diverse collection of recipes.

2. **Google Places API:** This API integration helps users find nearby restaurants based on location data. It provides comprehensive information about places, such as addresses and user reviews.

3. **Google Directions API:** This integration assists users in obtaining directions to a selected restaurant. The API provides detailed route information and distance calculations.

Backend Services: Firebase Firestore also acts as the backend database for storing information about delivery restaurants, such as their names, locations, and cuisines and information about user-uploaded recipes such as name, ingredients and preparation method.

Functional Requirements

1. Login

1.1 The user must be able to log in through an account

1.1.1 List of options

1.1.1.1 Register

1.1.1.2 Sign in

1.1.1.2.1 User must be able to sign in using their registered email and password.

1.1.1.2.2 The system must verify the login credentials.

1.1.1.3 Forget Password

1.1.1.3.1 User must be able to change their password through the system through their registered email address.

2. Register for new account

2.1 The user must be able to register for a new account

2.1.1 The account information includes:

2.1.1.1 Name

2.1.1.2 Email Address

2.1.1.2.1 The email address has to be unique in the system

2.1.1.3 Password that contains 8-12 characters, should only include alphabets, numbers and special characters. There should be at least one uppercase letter, one lowercase letter, one number and one special character.

2.1.2 The system must reconfirm with the user that the password is correct by asking the user to enter the password again (confirm password should be the same as the password)

2.1.3. The system must validate that all account information has been verified before finishing the registration process.

2.1.3.1 The system must send a verification link to the email address that the user entered and the user must verify their account to complete the registration process.

2.1.3.2 The system must validate if the email is already in the system.

3. Main Menu

3.1 The system must be able to display the list of options for users to use.

3.1.1 List of options available in the navigation bar

3.1.1.1 Home Page (which defaults on Recipe Section)

3.1.1.2 Eat Out Section

3.1.1.3 Delivery Section

3.1.1.4 Account Section

3.2 The system must allow users to access the home/recipe section.
3.3 The system must allow users to access the eat-out section.
3.4 The system must allow users to access the delivery section.
3.5 The system must allow users to access the account section.

4. Recipe Section

4.1 The user must be able to switch between sections that display a default list of recipes retrieved through the Recipes API (Official Recipes) and ones uploaded by app users (User Recipes)

4.2 The user must be able to search for recipes
- 4.2.1 The user must be able to search for recipes using the search function
- 4.2.2 The user must be able to filter recipes using the filter functions
  - 4.2.2.1 Filter options include:
    - 4.2.2.1.1 Cuisine Type
    - 4.2.2.1.2 Meal Type
    - 4.2.2.1.3 Difficulty

4.3 The system must be able to retrieve the list of recipes based on the user's search and filter.
- 4.3.1 Official Recipes should include:
  - 4.3.1.1 Recipe Name
  - 4.3.1.2 Cuisine
  - 4.3.1.3 Difficulty
  - 4.3.1.4 Calories
  - 4.3.1.5 Rating from 1 to 5
  - 4.3.1.6 Number of users who rated recipes
  - 4.3.1.7 Ingredients
  - 4.3.1.8 Preparation Time
  - 4.3.1.9 Instructions

- 4.3.2 User-Uploaded Recipes should include:
  - 4.3.2.1 Recipe Name
  - 4.3.2.2 Name of uploader
  - 4.3.2.3 Cuisine
  - 4.3.2.4 Difficulty
  - 4.3.2.5 Meal Type
  - 4.3.2.6 Rating from 1 to 5
  - 4.3.2.7 Number of users who rated recipes
  - 4.3.2.8 Description
  - 4.3.2.9 Calories
  - 4.3.2.10 Ingredients
  - 4.3.2.11 Instructions

4.4 The user must be able to rate user-uploaded recipes.

4.4.1 The user must be able to rate the recipe by giving the recipe a number of stars ranging from 1 to 5.

4.4 The user must be able to save user-uploaded recipes.

4.4.1 The user must be able to remove recipes from their saved list.

4.4.2 The user must not be able to save an already-saved recipe.

4.5 The user must be able to upload their own recipes.

4.5.1 Recipe uploaded should include:

4.5.1.1 Recipe name

4.5.1.2 Calories

4.5.1.3 Cooking time

4.5.1.4 Image of food item (not required)

4.5.1.5 Video of food item (not required)

4.5.1.6 Ingredients list

4.5.1.7 Preparation steps

4.5.1.8 Description

4.5.1.9 Cuisine type

4.5.1.10 Difficulty level

4.5.1.11 Meal type

5. Eat-Out Section

5.1 The user must be able to view a default list of restaurants on the eat-out main page

5.2 The user must provide location

5.2.1 The user must be able to manually input the address of the location

5.2.2 The system must be able to auto-detect the precise current location of the user's device if given location access

5.3 The system must be able to retrieve the list of restaurants within 2km radius of the location provided by user

5.3.1 The user must be able to see markers on the map for the nearby restaurants.

5.3.2 Restaurant details in eat-out should include:

5.3.2.1 Restaurant Name

5.3.2.2 Google Rating from 1 to 5

5.3.2.3 Opening hours

5.3.2.4 Location

5.3.2.5 Restaurant official website URL

5.3.2.6 Restaurant contact Number

5.3.2.7 Google Reviews

5.4 The user must be able to retrieve navigation directions to the restaurant

5.4.1 The user must be able to manually input the address of their initial

location

5.4.2 The user must be able to choose their mode of travel

5.4.2.1 Drive

5.4.2.2 Walk

5.4.2.3 Bicycle

5.4.3 The system must display directions, travel timing and travel distance for the chosen mode of travel.

6. Delivery Section

6.1 The user must be able to view a default list of restaurants on the delivery main page

6.2 The user must be able to search for restaurants for delivery

6.2.1 The user must be able to search for restaurants using the search function by either entering restaurant names or the cuisines offered by the restaurant

6.2.2 The user must be able to filter for restaurants by using the filter by rating function

6.2.2.1 The user can choose to view the restaurants according to their ratings, ordered either in ascending or descending order

6.2.2.2 The user can also choose to specify the rating as x and above (where x can be 1,2,3,4)

6.3 The system must be able to retrieve the list of restaurants based on the user's search and filter.

6.3.1 Restaurant details in delivery should include

6.3.3.1.1 Restaurant name

6.3.3.1.2 Rating

6.3.3.1.3 Number of reviews

6.3.3.1.4 Location

6.3.3.1.5 Average Delivery Time (in minutes)

6.3.3.1.6 Average Delivery Cost (in SGD)

6.3.3.1.7 Cuisines Served

6.3.3.1.8 Delivered by

6.4 The user must be able to save restaurants.

6.4.1 The user must be able to unsave a restaurant if they save it on the same page.

6.4.2 The user must be able to remove restaurants from their saved list.

7. Account Section

7.1 The user must be able to view their account details

7.1.1 The account details they can view include

7.1.1.1 Name
7.1.1.2 Email

7.2 The user must be able to view and manage their saved recipes
7.2.1 The user must be able to view list of saved recipes
7.2.2 The user must be able to view details of recipe by selecting the recipe
7.2.3 The user must be able to remove recipe from list of saved recipes

7.3 The user must be able to view and manage their saved restaurants
7.3.1 The user must be able to view list of saved restaurants
7.3.2 The user must be able to view details of restaurant by selecting the restaurant
7.3.3 The user must be able to remove restaurant from list of saved restaurants

7.4 The user must be able to manage their uploaded recipes
7.4.1 The user must be able to edit their recipe details.
7.4.2 The user must be able to delete their recipe.
7.4.3 The user must be able to upload a new recipe.

Non-Functional Requirements

1. Security
   - There should be hashing of passwords.

2. Reliability
   - Application has to maintain at least 99% uptime.

3. Performance
   - Application must load up within 3s of opening the app
   - The application must ensure a response time of under 3 seconds for any search query, recipe upload, or restaurant search operation, regardless of the number of concurrent users or the complexity of the request. This response time includes both server processing time and data retrieval time from external API's.
   - The application should be able to handle a certain minimum number of concurrent users performing search, upload, or viewing actions without experiencing any degradation in response time or service availability. This requirement ensures optimal performance and scalability under high user load scenarios.

4. Maintainability
   - Code maintainability, specify coding standards and practices to ensure the app's codebases are easy to maintain.

- Proper documentation by ensuring thorough documentation of the app's entire architecture.

5. Scalability
    - User-uploaded recipes can build up the recipe repository
    - Load handling, the app should be able to handle increased loads during peak usage times
    - Database scalability, database should scale to handle increased data volume
    - Database Indexing:
        - Properly design and maintain indexes on frequently queried columns.
        - Well-optimised indexes can significantly enhance query performance.
    - Caching:
        - Implement caching mechanisms to store frequently accessed data in memory.
        - Caching reduces the need to query the database for the same data repeatedly, improving performance.

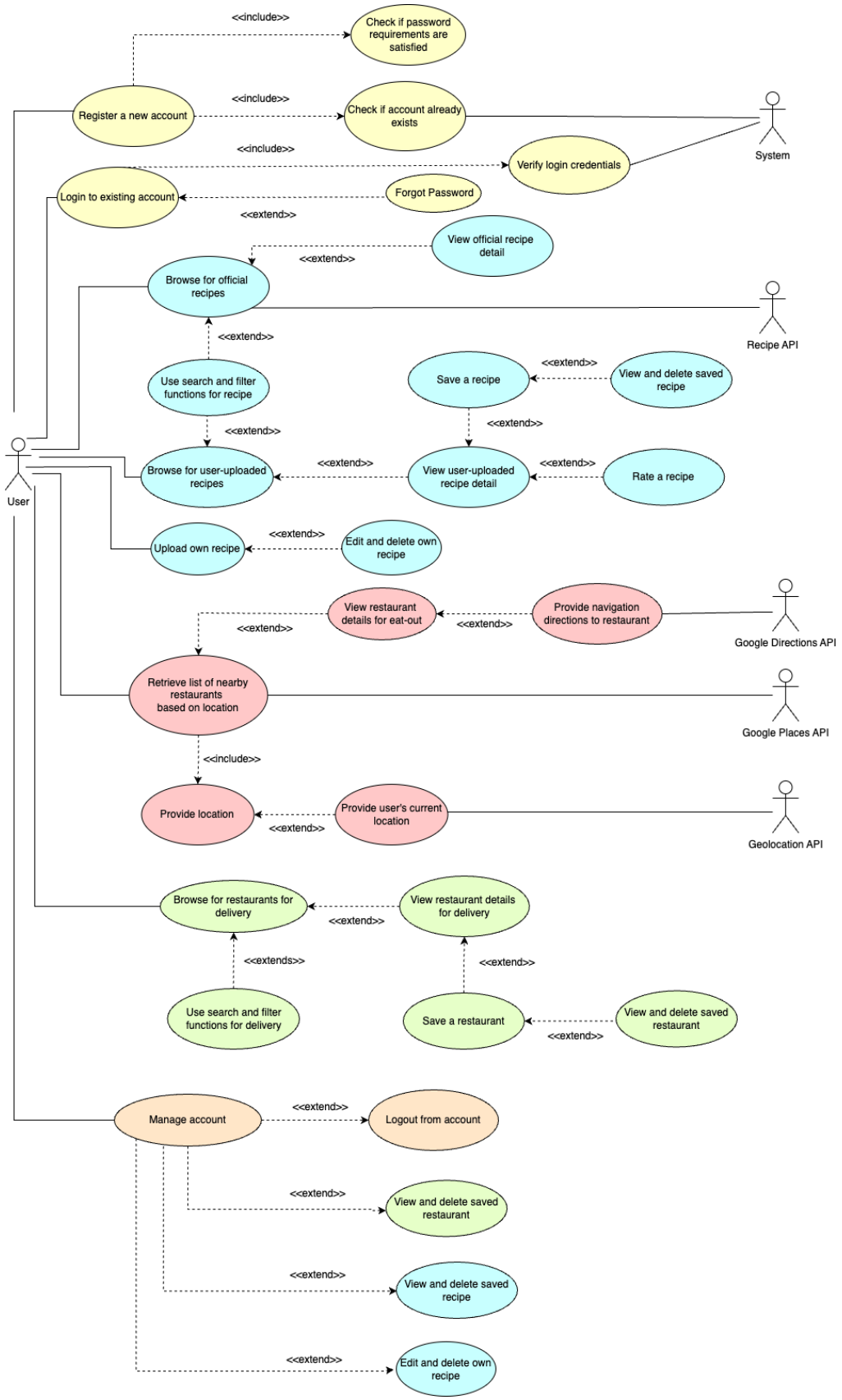
6. Usability
    1. Password should be strong (8 characters, 1 special character, 1 number)
    2. Friendly user interface designs by ensuring that app interface is intuitive and easy to use
    3. App is accessible to users with disabilities

Data Dictionary

Term	**Definition**
Recipe Section	A section within the application accessible to users. Recipes found within this section are either retrieved from our Recipe API or are uploaded by app users.The system displays a default set of recipes and users will also be able to search for recipes using search functions or filters. Recipes will contain information such as rating, cooking time, necessary ingredients and preparation method. Users can save and rate user-uploaded recipes and also view ratings given by other users.
Delivery Section	A section within the application accessible to users. The system displays a default set of restaurants for delivery and users will be able to search restaurants using search functions or filters. Restaurants under this section will contain information such as ratings, location, average delivery time and cost.

Eat-Out Section	A section within the application accessible to users. The system displays a default set of restaurants for delivery and users will be able to retrieve a list of nearby restaurants to the location they provide. Only restaurants that are registered on Google Places will be displayed to the user. Users can select the restaurant to view more information such as its Google ratings, reviews and location. Users can also get navigation directions to the restaurant.
Account	A virtual entity belonging to a user that gives them access to the services of Trinity. Users must possess and be signed in, in order to use any features and sections of the app. Users can create an account by registering with their name, a unique email and password that meets all our requirements.
Home Page	The main page that is displayed to the user upon starting the application. Users can navigate to their account settings or the 3 different sections through the navigation bar. The default section displayed to the user when the user opens the app is the Recipe Section.
Filter	Users are able to set filters for the type of recipes or delivery restaurants they want to view. Some of the filter options provided will be cuisine type, rating etc.
Search	Users are able to search for specific recipes or delivery restaurants. The system will return the closest match to the input.
Rating	Entries in the recipes and restaurants can be given a rating of 1 to 5 stars by users. The average rating will be displayed along with the total number of users who gave a rating for that recipe or restaurant.

Use Case Diagram



Use Case Description

Register and Login

Use Case ID:	1		
Use Case Name:	Register a new account		
Created By:	Jason Lim Jiasheng	Last Updated By:	Kalidoss Madhumitha
Date Created:	07/02/2024	Date Last Updated:	20/04/2024

Actor:	User (Initiating Actor), System
Description:	User registered a new email into the database of the System
Preconditions:	1. The user's email must be a valid email 2. The user's email must not already exist in the System's Database
Postconditions:	1. The user possess an email and password combination to login and access Trinity's services
Priority:	High
Frequency of Use:	0-1 per session
Flow of Events:	1. User selects the option to register for an account from the login page 2. System asks the user for a name, email, password (contains 8-12 characters, should only include alphabets, numbers and special characters. There should be at least one uppercase letter, one

	lowercase letter, one number and one special character) and password again to reconfirm 3. User inputs their name, email, password and a 'confirm password' field. 4. System checks if the email already exists in the Database using 'Check if account already exists' use case. 5. System checks if the password fulfils the requirements using 'Check if password requirements are satisfied' use case. 6. System checks if the 2 inputted passwords (password and 'confirm password') match 7. System informs user to verify their email by sending verification to the user's email 8. After confirmation of email, user is allowed to login through the 'Login to existing account' use case
Alternative Flows:	AF-S6: Passwords do not match 1. System outputs "Passwords do not match!" 2. Return to step 3
Exceptions:	-
Includes:	Check if password requirements are satisfied, Check if account already exists
Special Requirements:	-
Assumptions:	1. User has access to their email account and can complete email verification
Notes and Issues:	-

Use Case ID:	2
Use Case Name:	Check if account already exists

Created By:	Jason Lim Jiasheng	Last Updated By:	Kalidoss Madhumitha
Date Created:	07/02/2024	Date Last Updated:	20/04/2023

Actor:	User (Initiating Actor), System
Description:	System checks if email entered by user already has an account in the database
Preconditions:	1. User must enter a valid email account while trying to register a new account
Postconditions:	1. User is sent an email verification link
Priority:	Medium
Frequency of Use:	0-1 per session
Flow of Events:	1. User enters a valid email and fills up all the other fields and attempts to register 2. System checks its database for the email address 3. Account is created and email verification link is sent to user
Alternative Flows:	AF-S3: Email is already registered under a different account 1. Output: ”The email is already in use by a different account” 2. Return to step 1
Exceptions:	-
Includes:	-

Special Requirements:	-
Assumptions:	-
Notes and Issues:	-

Use Case ID:	3		
Use Case Name:	Check if password requirements are satisfied		
Created By:	Kalidoss Madhumitha	Last Updated By:	Kalidoss Madhumitha
Date Created:	20/04/2024	Date Last Updated:	20/04/2024

Actor:	User (Initiating Actor), System
Description:	System checks if password entered by user meets set requirements
Preconditions:	1. User must enter a valid email account in the email field 2. User must enter some password in the password field
Postconditions:	1. User is sent an email verification link
Priority:	Medium

Frequency of Use:	0-1 per session
Flow of Events:	1. User enters name, valid email and password in respective fields 2. System checks if password meets set requirements 3. Confirm password field matches with password field 4. Account is created and email verification link is sent to user
Alternative Flows:	AF-S2: Password does not fulfil requirements 1. System outputs a. “Minimum 8 characters required” if password has lesser than 8 character” b. “Maximum 12 characters accepted” if password has more than 12 characters c. “Should include 1) A digit 2) A lowercase character 3) An uppercase character 4) A special character” if password does not meet any one of the requirements 2. Return to step 1
Exceptions:	-
Includes:	-
Special Requirements:	-
Assumptions:	-
Notes and Issues:	-

Use Case ID:	4

Use Case Name:	Login to existing account		
Created By:	Jason Lim Jiasheng	Last Updated By:	Kalidoss Madhumitha
Date Created:	07/04/2024	Date Last Updated:	20/04/2023

Actor:	User (Initiating Actor), System
Description:	System validates the account credentials inputted by the user and allows the user to log in through an existing account
Preconditions:	1. User must input a valid email and password combination in the respective fields
Postconditions:	1. User is logged into their account and can access Trinity's services
Priority:	High
Frequency of Use:	Once per session
Flow of Events:	1. User inputs their account email and password into the respective fields. 2. The System verifies the email and the password combination with the database using 'Verify login credentials' use case 3. If the email and password is verified, the system authenticates the user that the login is successful
Alternative Flows:	-
Exceptions:	-

Includes:	Verify login credentials
Special Requirements:	-
Assumptions:	-
Notes and Issues:	-

Use Case ID:	5		
Use Case Name:	Verify login credentials		
Created By:	Jason Lim Jiasheng	Last Updated By:	Kalidoss Madhumitha
Date Created:	07/04/2024	Date Last Updated:	20/04/2023

Actor:	System
Description:	System validates the account credentials inputted by the user and allows the user to log in
Preconditions:	1. User must input a valid email and password combination in the respective fields
Postconditions:	1. User is logged into their account and can access Trinity's services
Priority:	High

Frequency of Use:	Once per session
Flow of Events:	1. User inputs their account email and password into the respective fields 2. The System verifies the email and the password combination with the database 3. If the email and password is verified, the system authenticates the user that the login is successful
Alternative Flows:	AF-S2: Email not in database 1. System output “Error: The supplied auth credential is incorrect, malformed or has expired” 2. Return to step 1 AF:S2: Wrong password 1. System output “Error: The supplied auth credential is incorrect, malformed or has expired” 2. Return to step 1
Exceptions:	-
Includes:	-
Special Requirements:	-
Assumptions:	-
Notes and Issues:	-

Use Case ID:	6
Use Case Name:	Forgot Password

Created By:	Jason Lim Jiasheng	Last Updated By:	Kalidoss Madhumitha
Date Created:	07/04/2024	Date Last Updated:	20/04/2023

Actor:	User (Initiating Actor), System
Description:	User forgets their password and requests for the System to reset their password.
Preconditions:	1. Account email must exist in the database
Postconditions:	1. Password is reset and user can use new password to log into their account
Priority:	Low
Frequency of Use:	0-1 per session
Flow of Events:	1. User enters a valid, registered email 2. System sends a password reset email to the email address 3. User logs into their email account and inputs a new password. 4. System authenticates that the account's password has been changed and returns the user to the 'Login to existing account' use case.
Alternative Flows:	-
Exceptions:	-
Includes:	-
Special Requirements:	-

Assumptions:	1. User has access to their email account and can retrieve password reset link from their email
Notes and Issues:	-

Recipes

Use Case ID:	7		
Use Case Name:	Browse for official recipes		
Created By:	Ananya	Last Updated By:	Ananya
Date Created:	07/02/2024	Date Last Updated:	12/02/2024

Actor:	User (Initiating actor), Recipe API
Description:	User looks for recipes which have been retrieved from an API. They can first browse through a default list of recipes which have been ordered randomly.
Preconditions:	1. User logs in and lands on the Recipes page (default Home Page), or User navigates to the Recipes main page using the navigation bar
Postconditions:	1. User is given a default list of recipes to view
Priority:	High

Frequency of Use:	0-10 times per session
Flow of Events:	1. User selects the recipe section to view 2. User chooses Official recipes from the tab (tab consists of Official Recipes and User Recipes) 3. User is given a list of official recipes to view 4. User can then choose to view the recipe details by clicking on a specific recipe
Alternative Flows:	-
Exceptions:	-
Includes:	-
Special Requirements:	-
Assumptions:	-
Notes and Issues:	-

Use Case ID:	8		
Use Case Name:	Browse for user-uploaded recipes		
Created By:	Ananya	Last Updated By:	Ananya
Date Created:	07/02/2024	Date Last Updated:	12/02/2024

Actor:	User
Description:	Users can look for recipes which have been retrieved from a recipes database which consists of all the recipes uploaded by app users. They can first browse through a default list of recipes which have been ordered randomly.
Preconditions:	1. User logs in and lands on the Recipes page (default Home Page), or User navigates to the Recipes main page using the navigation bar
Postconditions:	1. User is given a default list of recipes to view
Priority:	High
Frequency of Use:	0-10 times per session
Flow of Events:	1. User selects the recipe section to view 2. User chooses User recipes from the tab (tab consists of Official Recipes and User Recipes) 3. User is given a list of user-uploaded recipes to view 4. User can then choose to view the recipe details by clicking on a specific recipe
Alternative Flows:	-
Exceptions:	-
Includes:	-
Special Requirements:	-
Assumptions:	-
Notes and Issues:	-

Use Case ID:	9		
Use Case Name:	Use search and filter functions for recipes		
Created By:	Ananya	Last Updated By:	Ananya
Date Created:	07/02/2024	Date Last Updated:	12/02/2024

Actor:	User
Description:	Displays a list of recipes for the user, filtered based on: 1. User searches for the recipe using the search bar 2. User chooses filters from the options available a. Cuisine b. Meal Type c. Difficulty level
Preconditions:	1. User logs in and lands on the Recipes page (default Home Page), or User navigates to the Recipes main page using the navigation bar
Postconditions:	1. User is given a filtered list of recipes to view
Priority:	High
Frequency of Use:	0-10 times per session
Flow of Events:	1. User selects the recipe section to view 2. User is given a default list of recipes to view 3. User searches for a recipe/applies filters

	4. User is given a filtered list of recipes to view 5. User can then choose to view the recipe details by clicking on a specific recipe
Alternative Flows:	-
Exceptions:	EX-S4: No recipes fit the user's requirements 1. No recipe is displayed
Includes:	-
Special Requirements:	-
Assumptions:	The user can search for a recipe from either the official or user-uploaded recipes.
Notes and Issues:	-

Use Case ID:	10		
Use Case Name:	View user-uploaded recipe detail		
Created By:	Ananya	Last Updated By:	Ananya
Date Created:	07/02/2024	Date Last Updated:	13/02/2024

Actor:	User

Description:	Provides all the details of the user-uploaded recipes like: 1. Recipe name 2. Recipe picture 3. Recipe video 4. Name of uploader 5. Cuisine 6. Difficulty 7. Meal Type 8. Rating from 1 to 5 9. Number of users who rated recipes 10. Description/Extra Description 11. Calories 12. Ingredients 13. Instructions
Preconditions:	1. User chooses a specific recipe to view from the list of recipes available
Postconditions:	-
Priority:	High
Frequency of Use:	0-10 times per session
Flow of Events:	1. User chooses a recipe to view 2. System shows all details in the recipe
Alternative Flows:	-
Exceptions:	-
Includes:	-
Special Requirements:	-
Assumptions:	-

Notes and Issues:	-

Use Case ID:	11		
Use Case Name:	View official recipe detail		
Created By:	Ananya	Last Updated By:	Ananya
Date Created:	07/02/2024	Date Last Updated:	13/02/2024

Actor:	User
Description:	Provides all the details of the official recipes like: 1. Recipe name 2. Recipe picture 3. Cuisine 4. Difficulty 5. Calories 6. Meal Type 7. Rating from 1 to 5 8. Number of users who rated recipes 9. Ingredients 10. Preparation Time 11. Instructions
Preconditions:	1. User chooses a specific recipe to view from the list of recipes available
Postconditions:	-

Priority:	High
Frequency of Use:	0-10 times per session
Flow of Events:	1. User chooses a recipe to view 2. System shows all details in the recipe
Alternative Flows:	-
Exceptions:	-
Includes:	-
Special Requirements:	-
Assumptions:	-
Notes and Issues:	-

Use Case ID:	12		
Use Case Name:	Save a recipe		
Created By:	Ananya	Last Updated By:	Ananya
Date Created:	20/04/2024	Date Last Updated:	20/04/2024

Actor:	User

Description:	User can save a recipe to view later from the Accounts section
Preconditions:	1. User has not saved the recipe before 2. User chose a specific recipe to view and is on the details page
Postconditions:	1. Saved recipes can be accessed from the Accounts section
Priority:	Medium
Frequency of Use:	0-5 times per session
Flow of Events:	1. User chooses a recipe to view 2. System shows all details in the recipe, including a heart icon which the user can select to save the recipe 3. User clicks on the heart icon to save the recipe 4. Recipe gets saved and system displays ‘Added to favourites’ 5. User can view the saved recipe in the Accounts section
Alternative Flows:	AF-S4: User has already saved recipe 1. System output “Recipe already saved.”
Exceptions:	-
Includes:	-
Special Requirements:	-
Assumptions:	The recipe is a user-uploaded recipe
Notes and Issues:	-

Use Case ID:	13		
Use Case Name:	View and delete saved recipe		
Created By:	Ananya	Last Updated By:	Ananya
Date Created:	20/04/2024	Date Last Updated:	20/04/2024

Actor:	User
Description:	User can view their saved recipes from the Accounts Sections
Preconditions:	1. User went to the Accounts section from the navigation bar
Postconditions:	-
Priority:	Low
Frequency of Use:	0-2 times per session
Flow of Events:	1. User goes to Saved Recipes in the Accounts Section 2. User is displayed a list of saved recipes 3. User can view the details of the recipe by clicking on the specific recipe 4. User can remove the saved recipe by clicking on the ‘dustbin’ icon

Alternative Flows:	AF-S2: There are no saved recipes 1. Displays a blank page
Exceptions:	-
Includes:	-
Special Requirements:	-
Assumptions:	-
Notes and Issues:	-

Use Case ID:	14		
Use Case Name:	Rate a recipe		
Created By:	Ananya	Last Updated By:	Ananya
Date Created:	20/04/2024	Date Last Updated:	20/04/2024

Actor:	User
Description:	User can rate a recipe from 1 to 5 stars
Preconditions:	1. User chose a specific recipe to view and is on the details page

Postconditions:	-
Priority:	Medium
Frequency of Use:	0-5 times per session
Flow of Events:	1. User chooses a recipe to view 2. System shows all details in the recipe, including a 'Leave a rating' tab 3. On clicking on leave a rating, user gets the option to rate the recipe from 1 to 5 stars 4. User rates the recipe 5. The rating in the recipe details is updated accordingly
Alternative Flows:	-
Exceptions:	-
Includes:	-
Special Requirements:	-
Assumptions:	The recipe is a user-uploaded recipe
Notes and Issues:	-

Use Case ID:	15		
Use Case Name:	Upload own recipe		
Created By:	Ananya	Last Updated By:	Ananya

Date Created:	07/02/2024	Date Last Updated:	13/02/2024

Actor:	User
Description:	Allows for user to upload recipe information to be stored on the system's database
Preconditions:	1. User must be on the Recipes main page or on Manage my recipes page in Accounts section
Postconditions:	-
Priority:	Medium
Frequency of Use:	0-1 times per session
Flow of Events:	1. User selects the upload recipe option (+ button in the top right corner) in the Recipe main page or 'Upload Recipe' button on Manage my recipes page in Accounts section 2. System prompts user to enter a a. Recipe name b. Calories c. Cooking time d. Image of food item (not a compulsory field) e. Video of food item (not a compulsory field) f. Ingredients list g. Preparation steps h. Description i. Cuisine type j. Difficulty level k. Meal type 3. User fills up the required fields and clicks submit 4. System saves the recipe information and the uploader's data into the database

Alternative Flows:	AF-S3: Missing field 1. System prints out “Field required” 2. Continue from step 2
Exceptions:	-
Includes:	-
Special Requirements:	-
Assumptions:	-
Notes and Issues:	-

Use Case ID:	16		
Use Case Name:	Edit and delete own recipe		
Created By:	Ananya	Last Updated By:	Ananya
Date Created:	07/02/2024	Date Last Updated:	13/02/2024

Actor:	User
Description:	Allows for user to edit recipe details and also gives the option to delete the recipe

Preconditions:	1. User must be on the Accounts Section
Postconditions:	-
Priority:	Medium
Frequency of Use:	0-1 times per session
Flow of Events:	1. User selects ‘Manage my recipes’ from the Accounts page 2. System displays a list of the recipes uploaded by that user 3. User chooses the recipe they want to edit/delete 4. System displays the recipe information and provides the user the option to edit the relevant fields a. User edits the recipe and clicks on ‘Edit Recipe’ button b. User clicks on ‘Delete Recipe’ button 5. Based on the above selection a. If edited, the updated recipe is stored in the database b. If deleted, the recipe is removed from the database
Alternative Flows:	AF-S2: No uploaded recipes 1. System displays an empty list with the option for the user to upload a new recipe
Exceptions:	-
Includes:	-
Special Requirements:	-
Assumptions:	-
Notes and Issues:	-

Eat-Out

Use Case ID:	17		
Use Case Name:	Retrieve list of nearby restaurants based on location		
Created By:	Ananya	Last Updated By:	Ananya
Date Created:	07/02/2024	Date Last Updated:	13/02/2024

Actor:	User, Google Places API
Description:	To retrieve the list of restaurants from the the Google Places API based on the location provided
Preconditions:	1) User must navigate to Eat-Out section from the Navigation Bar
Postconditions:	-
Priority:	High
Frequency of Use:	0-10 per session
Flow of Events:	1. User taps on the Eat Out option 2. System provides a random list of restaurants available 3. User taps on 'Find More Restaurants' and is directed to the page where they input a location 4. Based on the location provided, System retrieves a list of nearby restaurants (within 2km radius of the location) from the Google Places API and displays it as markers on the map

	5. User can tap on individual markers to see the names of restaurants 6. User can also click on 'View List of Restaurants' to get a whole list of these available restaurants nearby
Alternative Flows:	-
Exceptions:	-
Includes:	Provide Location
Special Requirements:	-
Assumptions:	-
Notes and Issues:	-

Use Case ID:	18		
Use Case Name:	Provide Location		
Created By:	Ananya	Last Updated By:	Ananya
Date Created:	07/02/2024	Date Last Updated:	13/02/2024

Actor:	User

Description:	To ask the user to input a location to retrieve the list of restaurants from the the Google Places API based on the location provided
Preconditions:	1. User must navigate to Eat-Out section from the Navigation Bar 2. User must select 'Find more restaurants'
Postconditions:	-
Priority:	High
Frequency of Use:	0-10 per session
Flow of Events:	1. User taps on 'Find More Restaurants' button 2. User chooses their location 3. User taps on Search button 4. System retrieves a list of nearby restaurants (within 2km radius of the location) from the Google Places API and displays it as markers on the map
Alternative Flows:	-
Exceptions:	-
Includes:	-
Special Requirements:	-
Assumptions:	-
Notes and Issues:	-

Use Case ID:	19		
Use Case Name:	Provide User's Current Location		
Created By:	Ananya	Last Updated By:	Ananya
Date Created:	7/2/2024	Date Last Updated:	13/2/2024

Actor:	User (Initiating actor), Geolocation API
Description:	To ask the user to access their current location to retrieve the list of restaurants from the the Google Places API based on their location
Preconditions:	1. User must navigate to Eat-Out section from the Navigation Bar 2. User must select 'Find more restaurants'
Postconditions:	-
Priority:	High
Frequency of Use:	0-10 per session
Flow of Events:	1. User taps on 'Find More Restaurants' button 2. User taps on Use Current Location 3. System retrieves a list of nearby restaurants (within 2km of the location) from the Google Places API and displays it as markers on the map
Alternative Flows:	-

Exceptions:	-
Includes:	-
Special Requirements:	-
Assumptions:	-
Notes and Issues:	-

Use Case ID:	20		
Use Case Name:	View restaurant details for eat-out		
Created By:	Ananya	Last Updated By:	Ananya
Date Created:	07/02/2024	Date Last Updated:	13/02/2024

Actor:	User
Description:	Provides all the details of the restaurants like: 1. Restaurant name 2. Restaurant picture 3. Google Rating from 1 to 5 4. Opening hours 5. Location 6. Restaurant official website URL

	7. Restaurant contact Number 8. Google Reviews
Preconditions:	1. User clicks on a specific restaurant from the list provided
Postconditions:	-
Priority:	High
Frequency of Use:	0-10 times per session
Flow of Events:	1. User chooses a restaurant to view 2. System shows all details of the restaurant
Alternative Flows:	-
Exceptions:	-
Includes:	-
Special Requirements:	-
Assumptions:	-
Notes and Issues:	-

Use Case ID:	21
Use Case Name:	Provide navigation directions to restaurant

Created By:	Ananya	Last Updated By:	Ananya
Date Created:	07/02/2024	Date Last Updated:	13/02/2024

Actor:	Google Directions API
Description:	Provides navigation to the restaurant from the location the user provided. The user can choose any one of the travel modes: 1. Drive 2. Walk 3. Bicycle
Preconditions:	1. User clicks on a specific restaurant from the list provided 2. User clicks on the address from the details page
Postconditions:	-
Priority:	Medium
Frequency of Use:	0-5 times per session
Flow of Events:	1. User chooses a restaurant to view 2. System shows all details of the restaurant, including the address of the restaurant 3. User clicks on the address of restaurant 4. User selects a starting location and chooses their mode of travel 5. The system must display directions, travel timing and travel distance for the chosen mode of travel.
Alternative Flows:	-
Exceptions:	-

Includes:	-
Special Requirements:	-
Assumptions:	-
Notes and Issues:	-

Delivery

Use Case ID:	22		
Use Case Name:	Browse for restaurants for delivery		
Created By:	Kalidoss Madhumitha	Last Updated By:	Kalidoss Madhumitha
Date Created:	20/04/2024	Date Last Updated:	20/04/2024

Actor:	User (Initiating actor), System
Description:	User looks for restaurants available for delivery which have been retrieved from the database. They can first browse through a default list of restaurants which have been ordered randomly.
Preconditions:	1. User logs in and navigates to the delivery main page using the navigation bar

Postconditions:	1. User is given a default list of restaurants for delivery to view
Priority:	High
Frequency of Use:	0-10 times per session
Flow of Events:	1. User selects the delivery section to view 2. User is given a list of restaurants for delivery to view 3. User can then choose to view the restaurant details by clicking on a specific restaurant
Alternative Flows:	-
Exceptions:	-
Includes:	-
Special Requirements:	-
Assumptions:	-
Notes and Issues:	-

Use Case ID:	23		
Use Case Name:	Use search and filter functions for delivery		
Created By:	Kalidoss Madhumitha	Last Updated By:	Kalidoss Madhumitha

Date Created:	20/04/2024	Date Last Updated:	20/04/2024

Actor:	User
Description:	Displays a list of restaurants for delivery for the user, filtered based on: 1. User searches for the restaurant name or cuisines offered in restaurant using the search bar 2. User applies rating filters from the options available a. Sort in descending order (default) b. Sort in ascending order c. Specify the rating as x and above (where x can be 1,2,3,4)
Preconditions:	1. User logs in and navigates to the Delivery main page using the navigation bar
Postconditions:	1. User is given a filtered list of restaurants for delivery to view
Priority:	High
Frequency of Use:	0-10 times per session
Flow of Events:	1. User selects the delivery section to view 2. User is given a default list of restaurants for delivery to view 3. User searches for a restaurant/applies filters 4. User is given a filtered list of restaurants to view 5. User can then choose to view the restaurant details by clicking on a specific restaurant
Alternative Flows:	-
Exceptions:	EX-S4: No restaurants fit the user's requirements

	1. No restaurant is displayed
Includes:	-
Special Requirements:	-
Assumptions:	-
Notes and Issues:	-

Use Case ID:	24		
Use Case Name:	View restaurant details for delivery		
Created By:	Kalidoss Madhumitha	Last Updated By:	Kalidoss Madhumitha
Date Created:	20/04/2024	Date Last Updated:	20/04/2024

Actor:	User
Description:	Provides all the details of restaurants for delivery like: 1. Restaurant name 2. Restaurant picture 3. Rating 4. Number of reviews 5. Location 6. Average Delivery Time (in minutes) 7. Average Delivery Cost (in SGD) 8. Cuisines Served 9. Delivered by

Preconditions:	1. User chooses a specific restaurant to view from the list of restaurants available
Postconditions:	-
Priority:	High
Frequency of Use:	0-10 times per session
Flow of Events:	1. User chooses a restaurant to view 2. System shows all details of the restaurant
Alternative Flows:	-
Exceptions:	-
Includes:	-
Special Requirements:	-
Assumptions:	-
Notes and Issues:	-

Use Case ID:	25		
Use Case Name:	Save a restaurant		
Created By:	Kalidoss Madhumitha	Last Updated By:	Kalidoss Madhumitha

Date Created:	20/04/2024	Date Last Updated:	20/04/2024

Actor:	User
Description:	User can save a restaurant for delivery to view later from the Accounts section
Preconditions:	1. User chose a specific restaurant to view and is on the details page
Postconditions:	1. Saved restaurants can be accessed from the Accounts section
Priority:	Medium
Frequency of Use:	0-5 times per session
Flow of Events:	1. User chooses a restaurant for delivery to view 2. System shows all details of the restaurant, including a heart icon which the user can select to save the restaurant 3. User clicks on the heart icon to save the restaurant 4. Restaurant gets saved and system displays ‘Added to favourites’ 5. User can view the saved restaurant in the Accounts section
Alternative Flows:	-
Exceptions:	-
Includes:	-
Special Requirements:	-

Assumptions:	-
Notes and Issues:	-

Use Case ID:	26		
Use Case Name:	View and delete saved restaurant		
Created By:	Kalidoss Madhumitha	Last Updated By:	Kalidoss Madhumitha
Date Created:	20/04/2024	Date Last Updated:	20/04/2024

Actor:	User
Description:	User can view their saved restaurants from the Accounts section
Preconditions:	1. User went to the Accounts section from the navigation bar
Postconditions:	-
Priority:	Low
Frequency of Use:	0-2 times per session
Flow of Events:	1. User goes to Saved Restaurants in the Accounts Section 2. User is displayed a list of saved restaurants

	3. User can view details of the restaurant by clicking on the specific restaurant 4. User can remove the saved restaurant by clicking on the ‘dustbin’ icon
Alternative Flows:	AF-S2: There are no saved restaurants 1. Displays a blank page
Exceptions:	-
Includes:	-
Special Requirements:	-
Assumptions:	-
Notes and Issues:	-

Manage Account

Use Case ID:	27		
Use Case Name:	Manage account		
Created By:	Kalidoss Madhumitha	Last Updated By:	Kalidoss Madhumitha
Date Created:	20/04/2024	Date Last Updated:	20/04/2024

Actor:	User (Initiating Actor), System

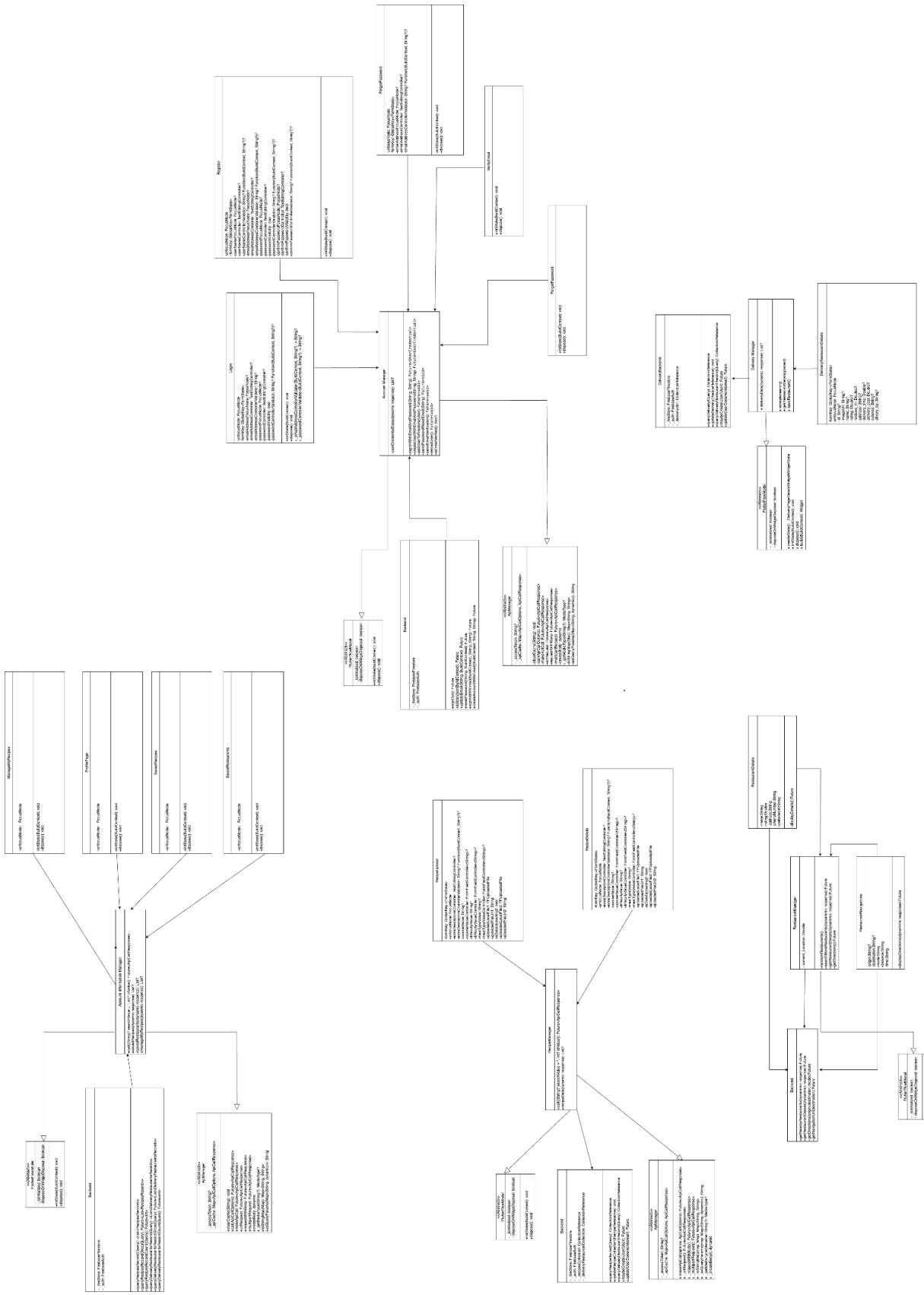
Description:	Allows for the user to manage their profile. They can view and manage their saved restaurants, saved recipes, uploaded recipes and also logout from their account.
Preconditions:	1. User must have an account
Postconditions:	-
Priority:	Medium
Frequency of Use:	0 to 5 times per session
Flow of Events:	1. User goes to the profile section 2. System displays the options to user a. If the user taps “Saved Restaurants”, the ‘View and delete saved restaurant’ use case is called b. If the user taps “Saved Recipes”, the ‘View and delete saved recipe’ use case is called c. If the user taps “Manage my recipes”, the ‘Edit and delete own recipes’ use case is called
Alternative Flows:	-
Exceptions:	-
Includes:	n
Special Requirements:	-
Assumptions:	-
Notes and Issues:	-

Use Case ID:	28		
Use Case Name:	Logout from account		
Created By:	Kalidoss Madhumitha	Last Updated By:	Kalidoss Madhumitha
Date Created:	27/2/2024	Date Last Updated:	20/4/2024

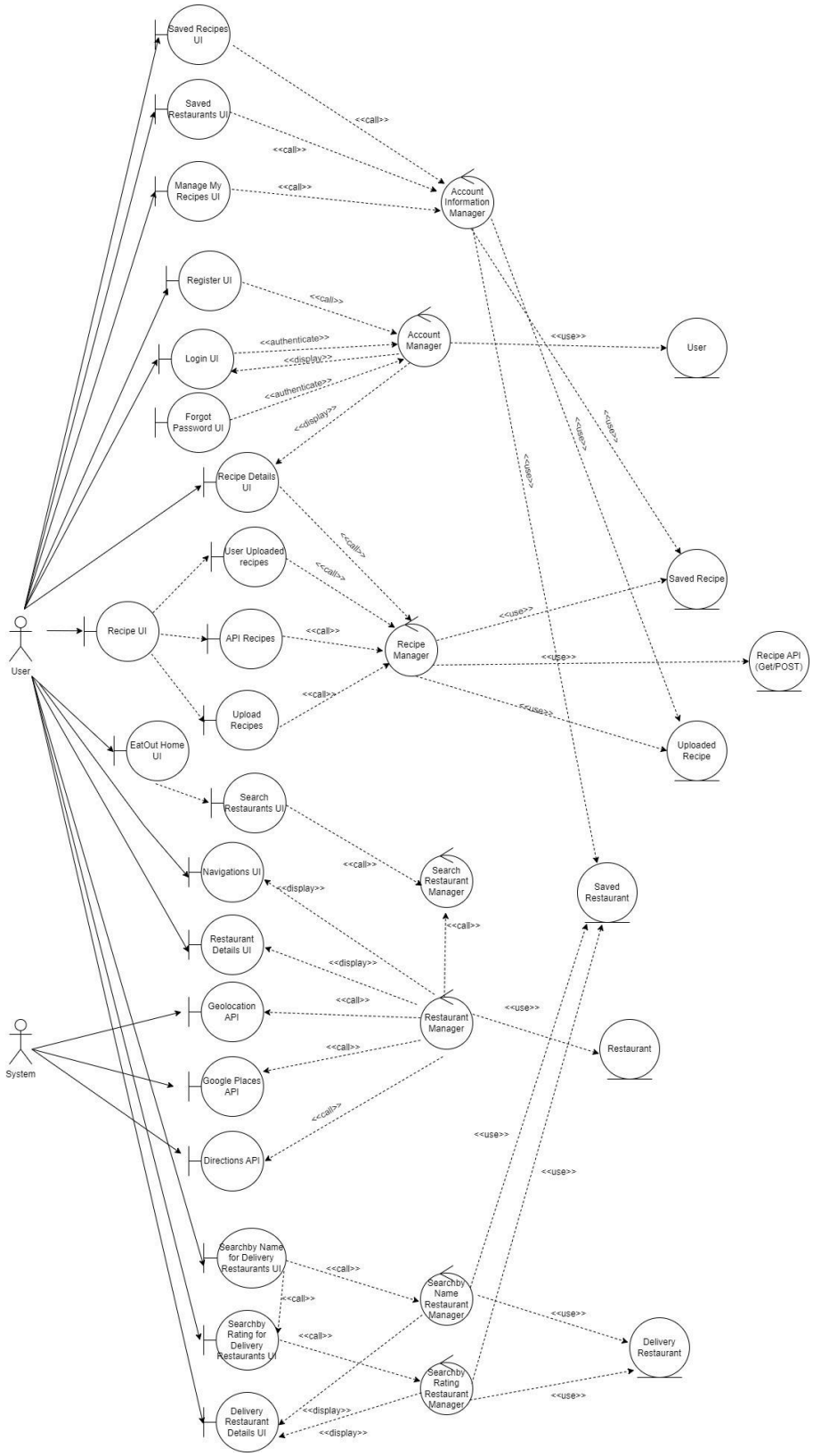
Actor:	User
Description:	User can logout from their account
Preconditions:	User must be logged in to their account
Postconditions:	User is logged out of their account
Priority:	Medium
Frequency of Use:	0-1 time per session
Flow of Events:	1. User clicks on the logout button 2. System logs out user from their account 3. User is directed to the login page
Alternative Flows:	-
Exceptions:	-
Includes:	-
Special Requirements:	-

Assumptions:	-
Notes and Issues:	-

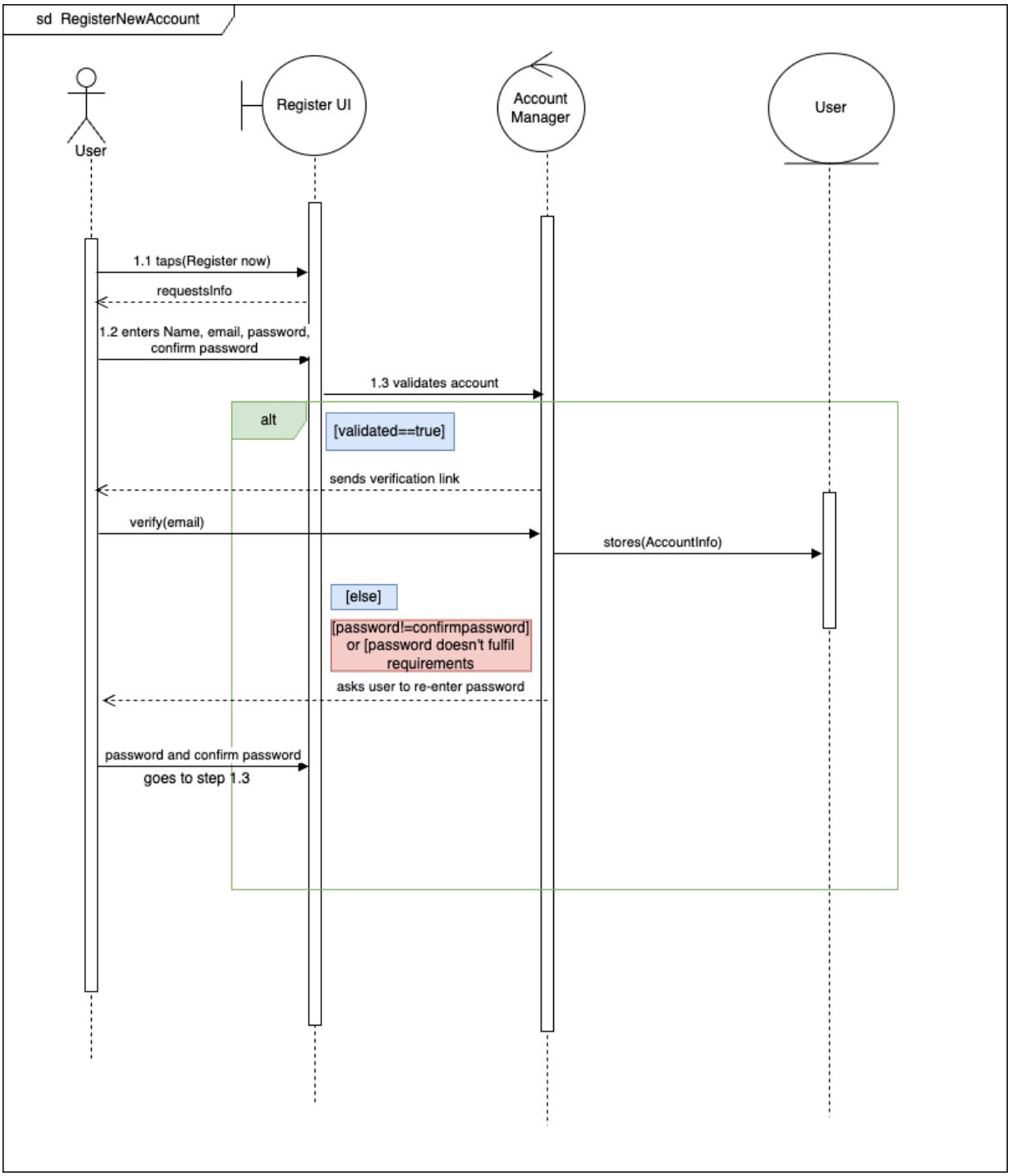
Class Diagram

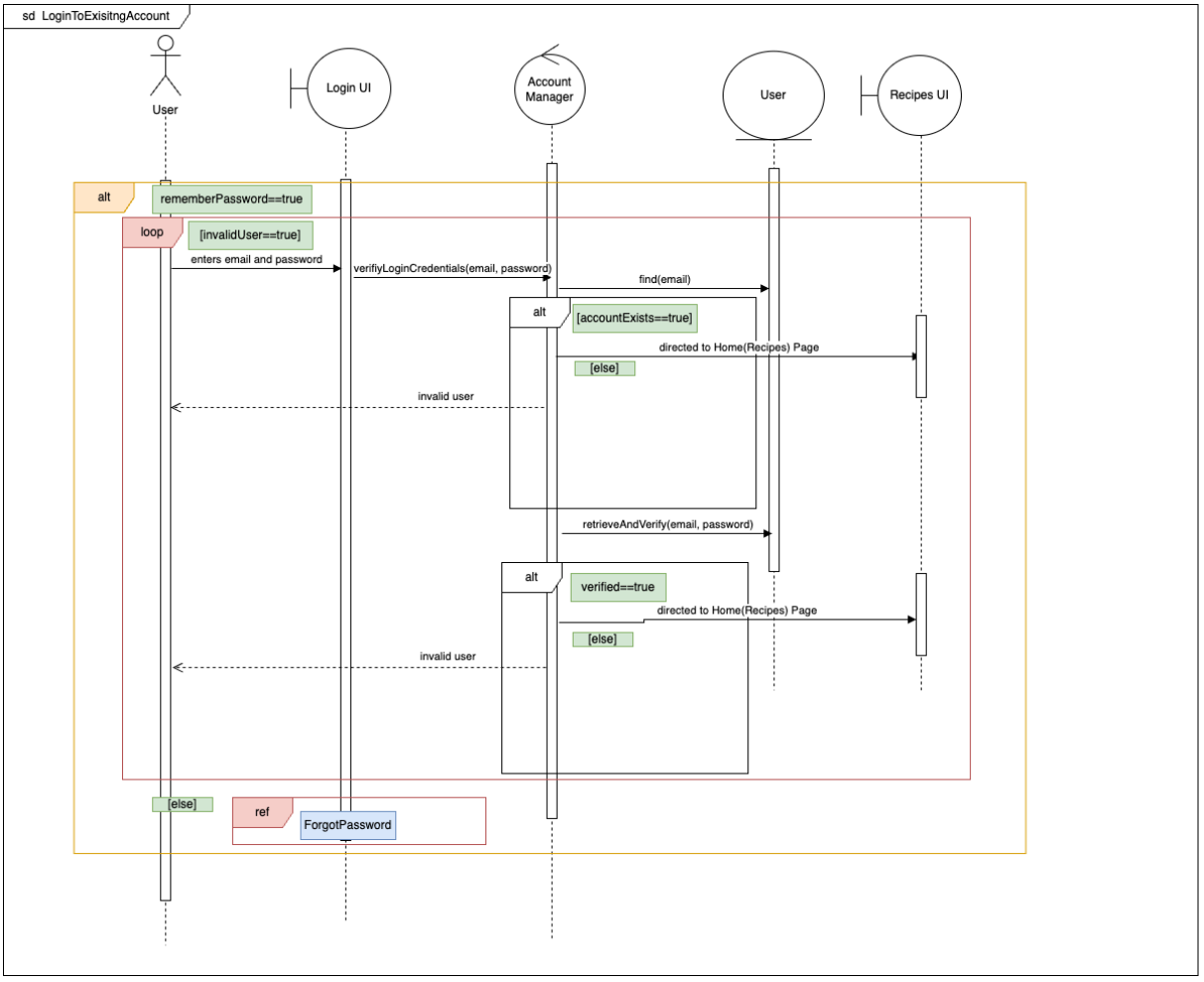


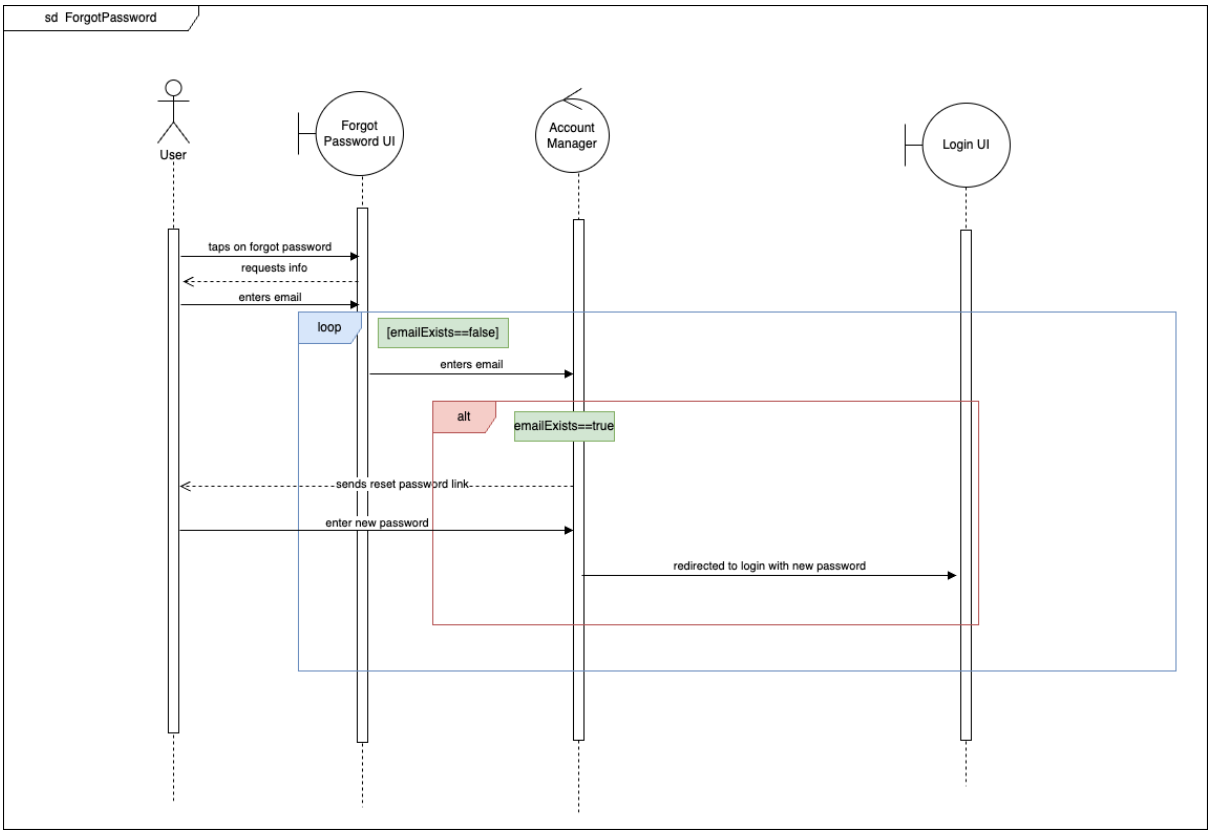
ECB Diagram

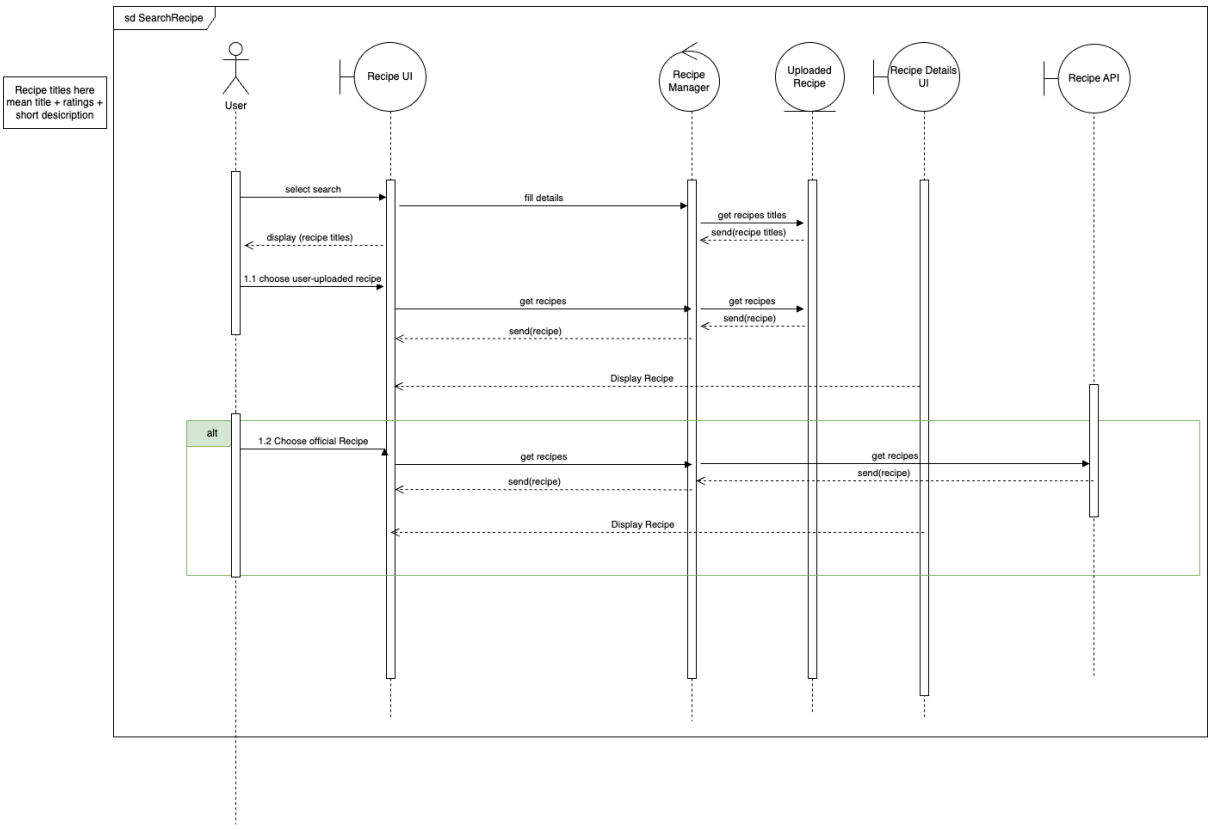

Saved Recipes UI
Saved Restaurants UI
Manage My Recipes UI
Account Information Manager
Register UI
Login UI
Forgot Password UI
Account Manager
User
Recipe Details UI
User Uploaded recipes
Saved Recipe
Recipe UI
API Recipes
Recipe Manager
Recipe API (Get/POST)
Upload Recipes
Uploaded Recipe
EatOut Home UI
Search Restaurants UI
Navigations UI
Search Restaurant Manager
Saved Restaurant
Restaurant Details UI
Geolocation API
Restaurant Manager
Restaurant
Google Places API
Directions API
Searchby Name for Delivery Restaurants UI
Searchby Name Restaurant Manager
Searchby Rating for Delivery Restaurants UI
Searchby Rating Restaurant Manager
Delivery Restaurant
Delivery Restaurant Details UI
User
System
<<call>>
<<use>>
<<authenticate>>
<<display>>

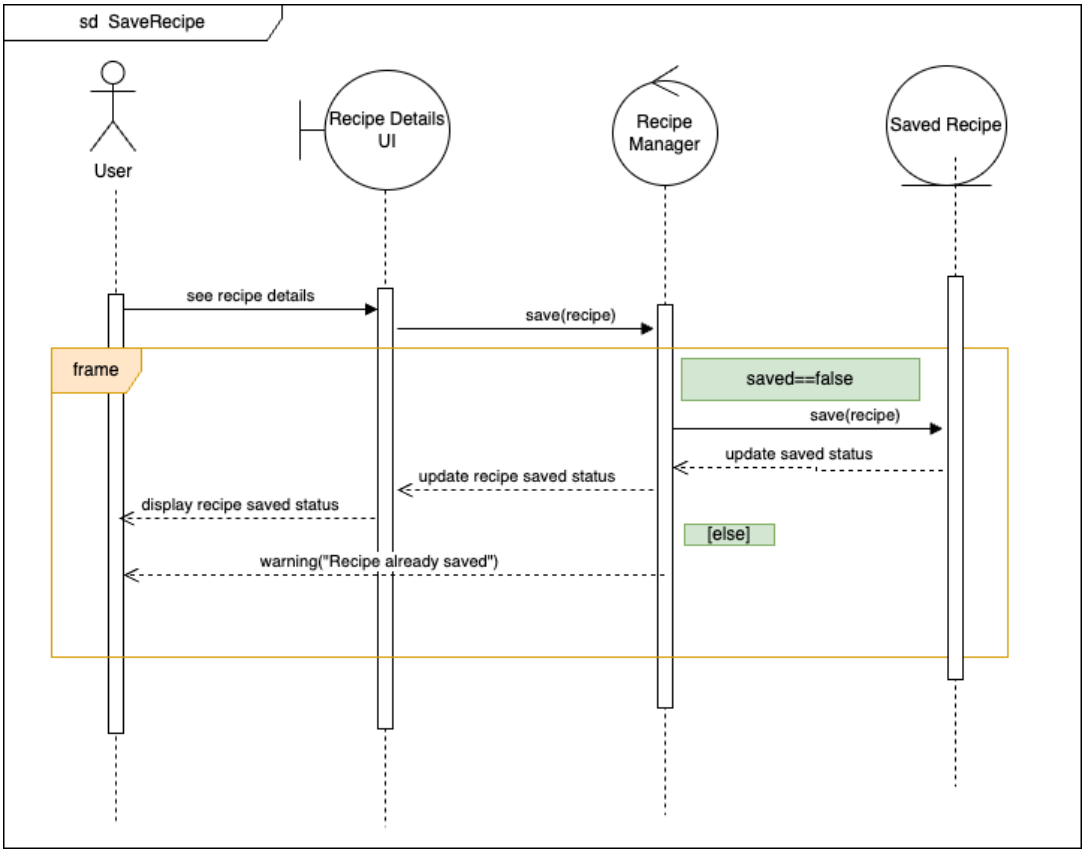
Sequence Diagram

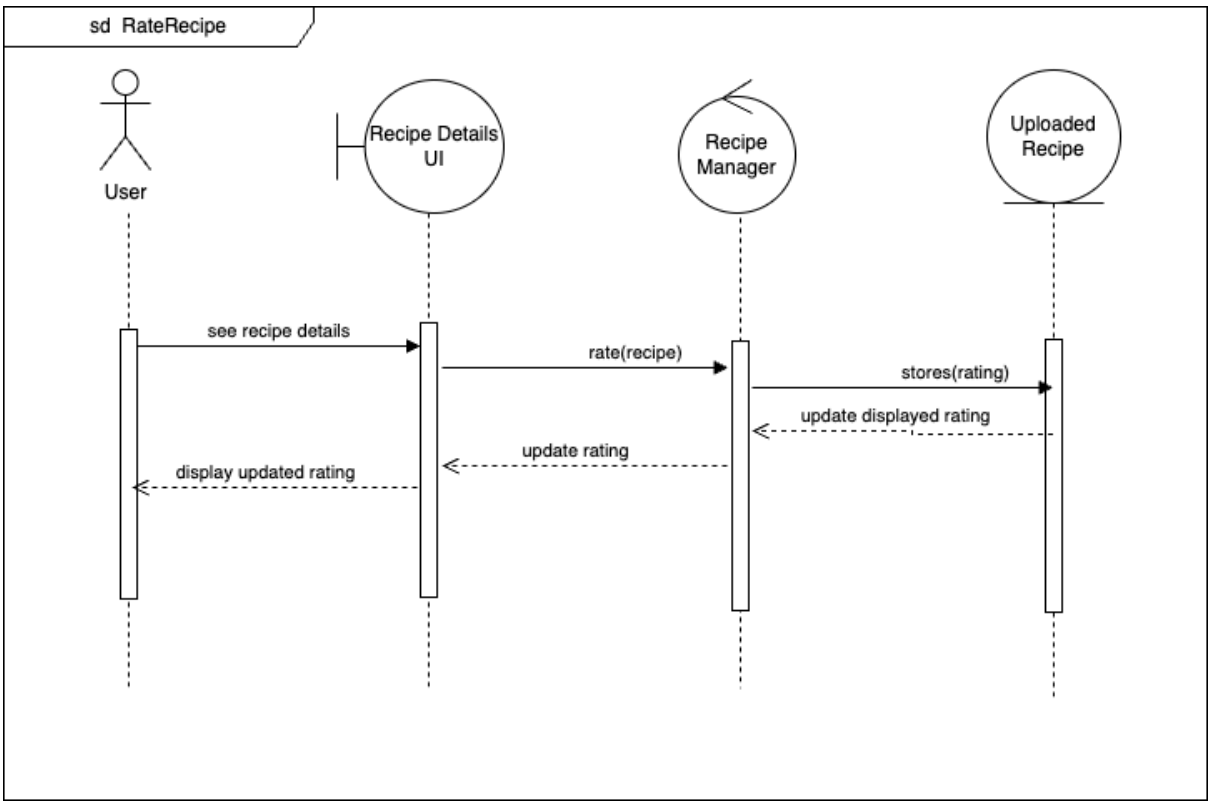


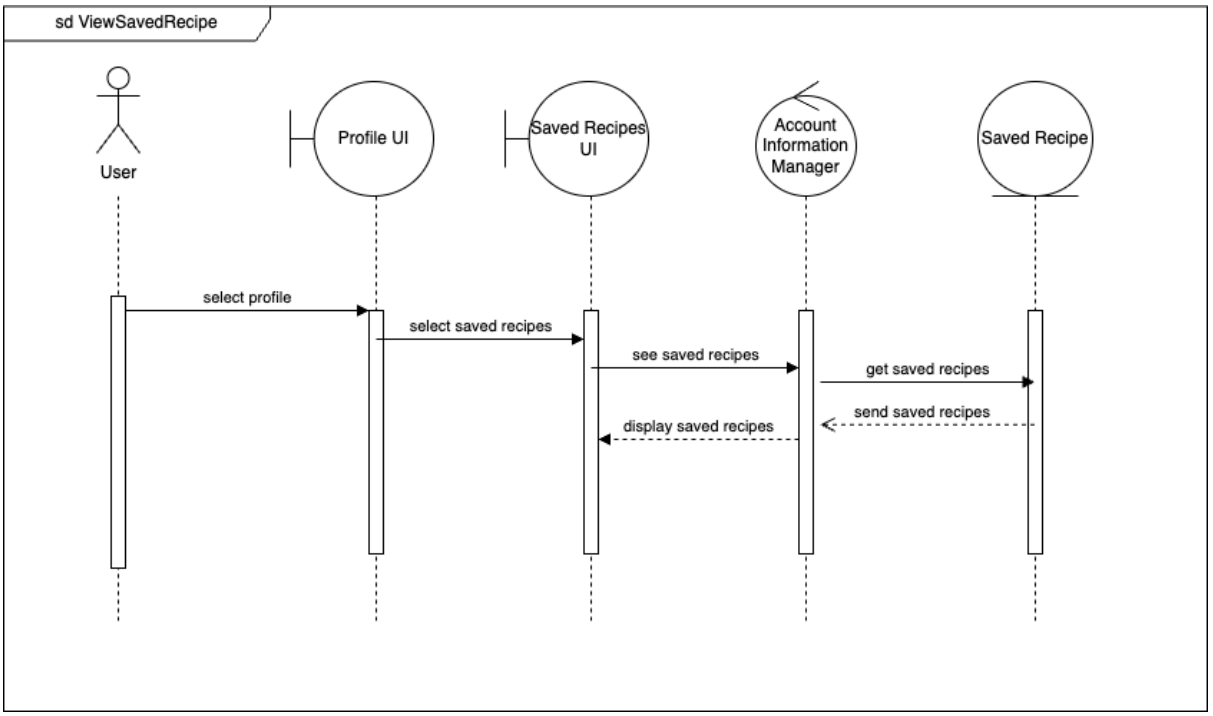

sd LoginToExisitngAccount
User
Login UI
Account Manager
User
Recipes UI
alt
rememberPassword==true
loop
[invalidUser==true]
enters email and password
verifiyLoginCredentials(email, password)
find(email)
alt
[accountExists==true]
directed to Home(Recipes) Page
[else]
invalid user
retrieveAndVerify(email, password)
alt
verified==true
directed to Home(Recipes) Page
[else]
invalid user
[else]
ref
ForgotPassword

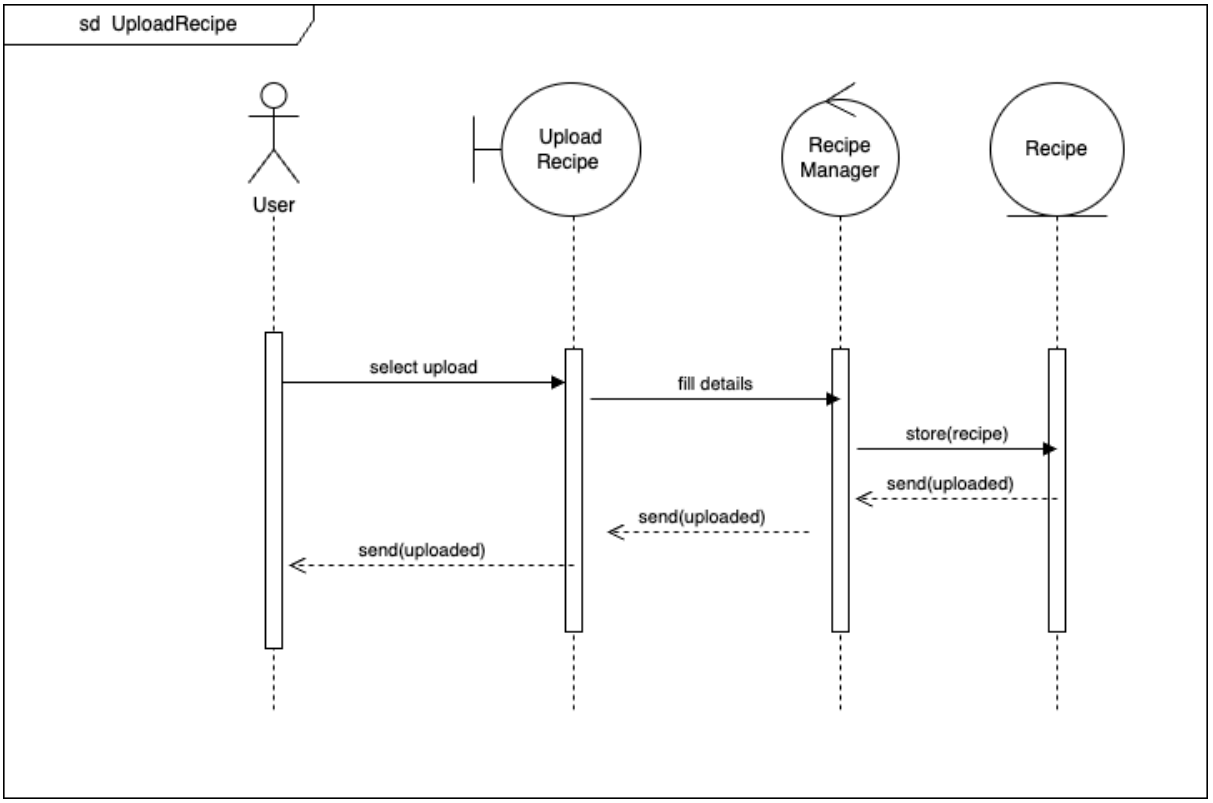

sd ForgotPassword
User
Forgot Password UI
Account Manager
Login UI
taps on forgot password
requests info
enters email
loop
[emailExists==false]
enters email
alt
emailExists==true
sends reset password link
enter new password
redirected to login with new password

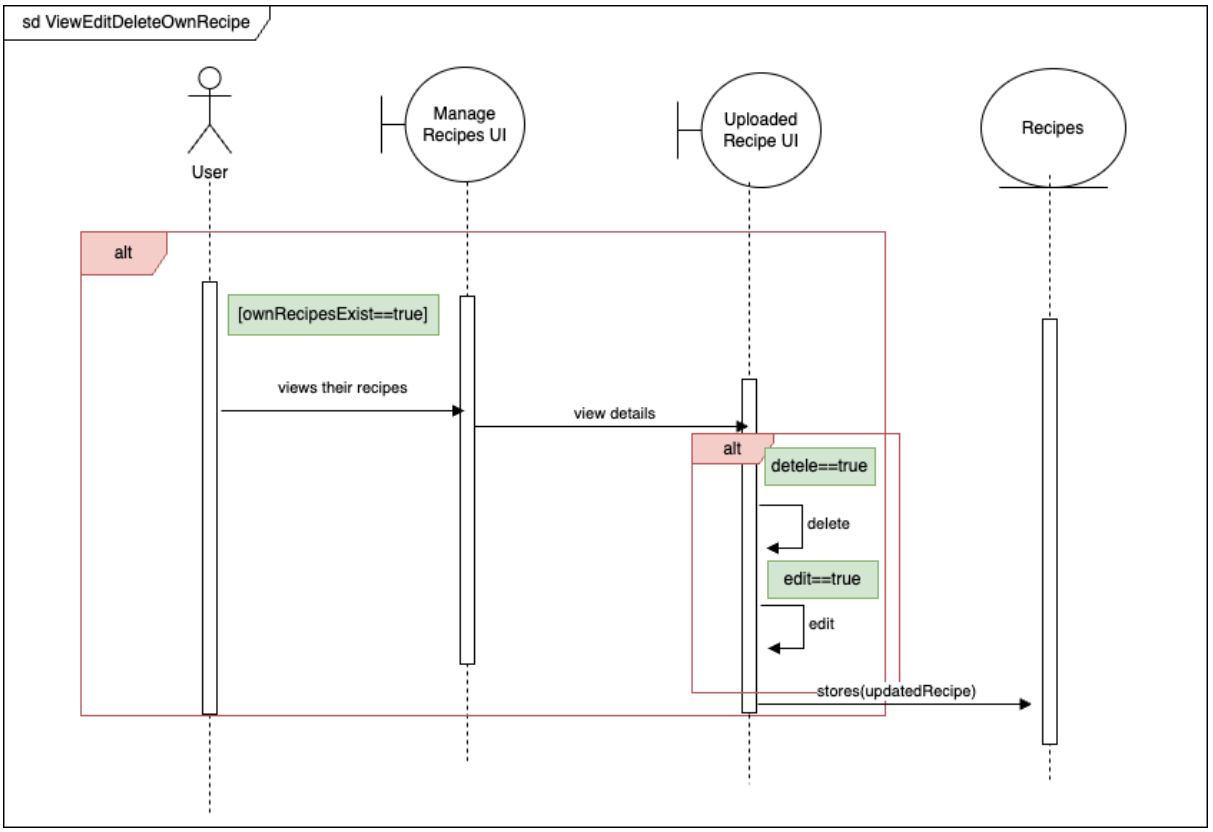

sd SearchRecipe
Recipe titles here mean title + ratings + short desicription
User
Recipe UI
Recipe Manager
Uploaded Recipe
Recipe Details UI
Recipe API
select search
fill details
get recipes titles
send(recipe titles)
display (recipe titles)
1.1 choose user-uploaded recipe
get recipes
get recipes
send(recipe)
send(recipe)
Display Recipe
alt
1.2 Choose official Recipe
get recipes
get recipes
send(recipe)
send(recipe)
Display Recipe

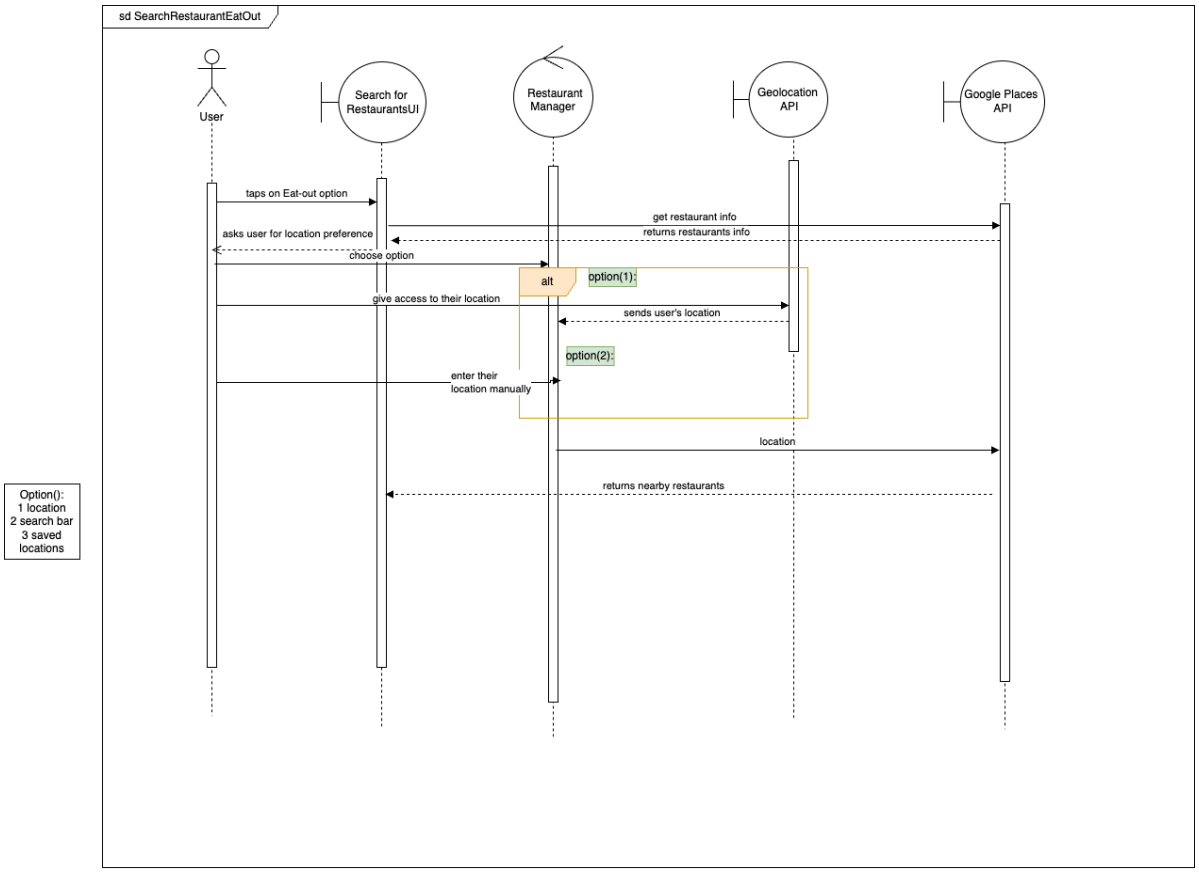

sd SaveRecipe
User
Recipe Details UI
Recipe Manager
Saved Recipe
see recipe details
save(recipe)
frame
saved==false
save(recipe)
update saved status
update recipe saved status
display recipe saved status
[else]
warning("Recipe already saved")

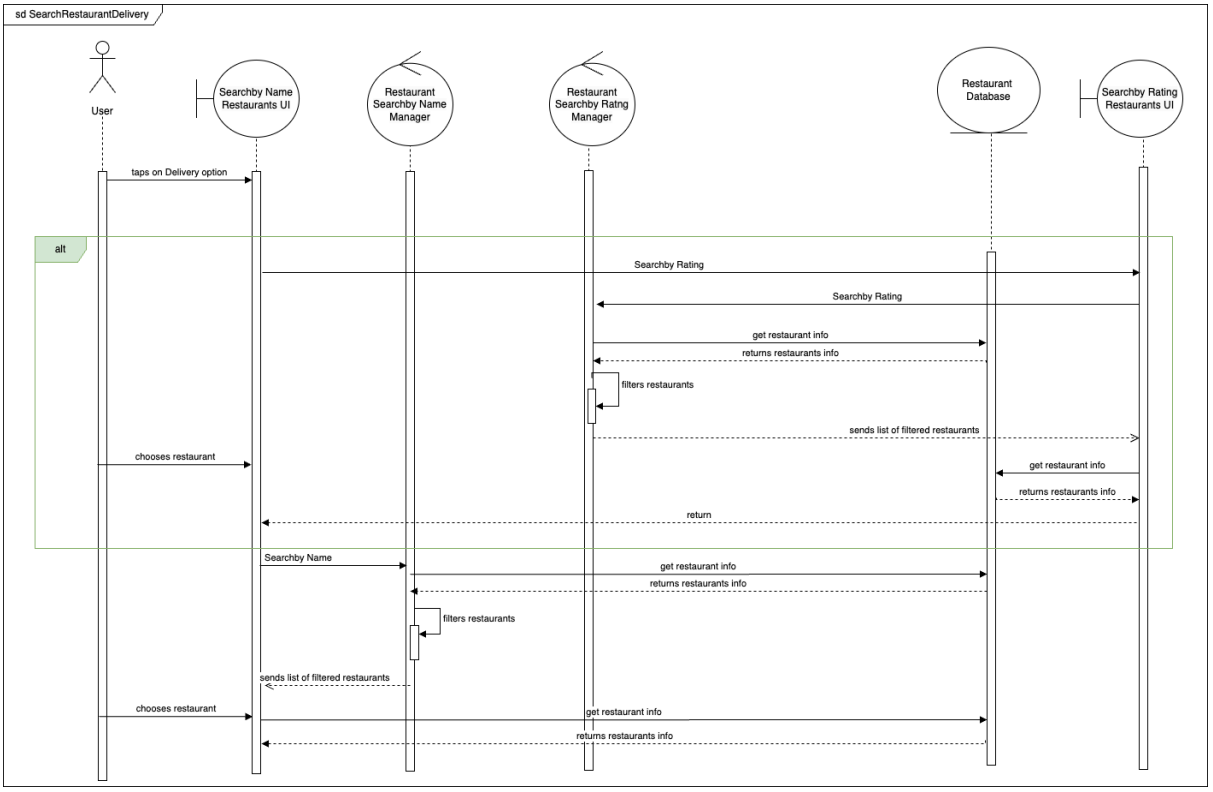

sd RateRecipe
User
Recipe Details UI
Recipe Manager
Uploaded Recipe
see recipe details
rate(recipe)
stores(rating)
update displayed rating
update rating
display updated rating

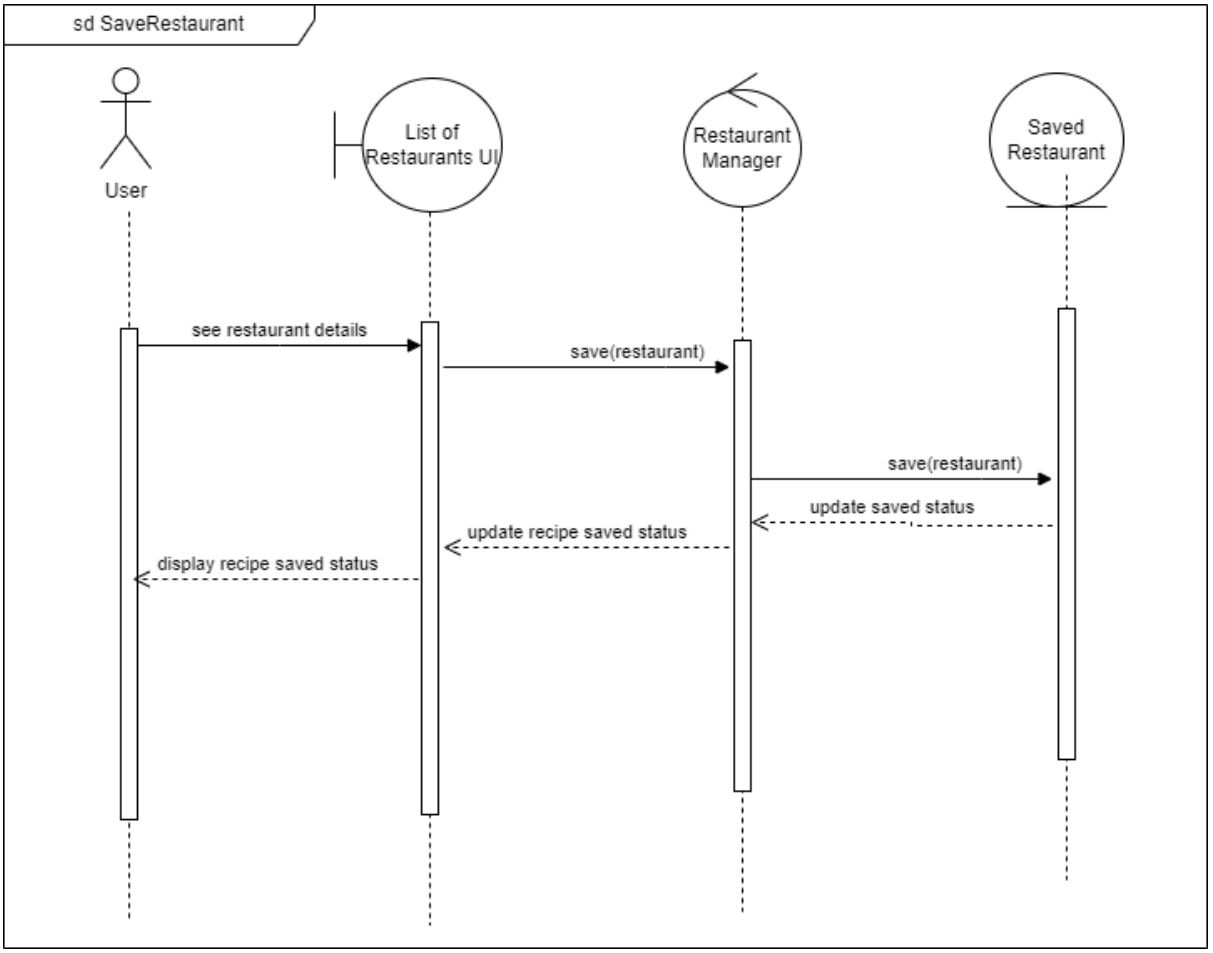

sd ViewSavedRecipe
User
Profile UI
Saved Recipes UI
Account Information Manager
Saved Recipe
select profile
select saved recipes
see saved recipes
get saved recipes
send saved recipes
display saved recipes

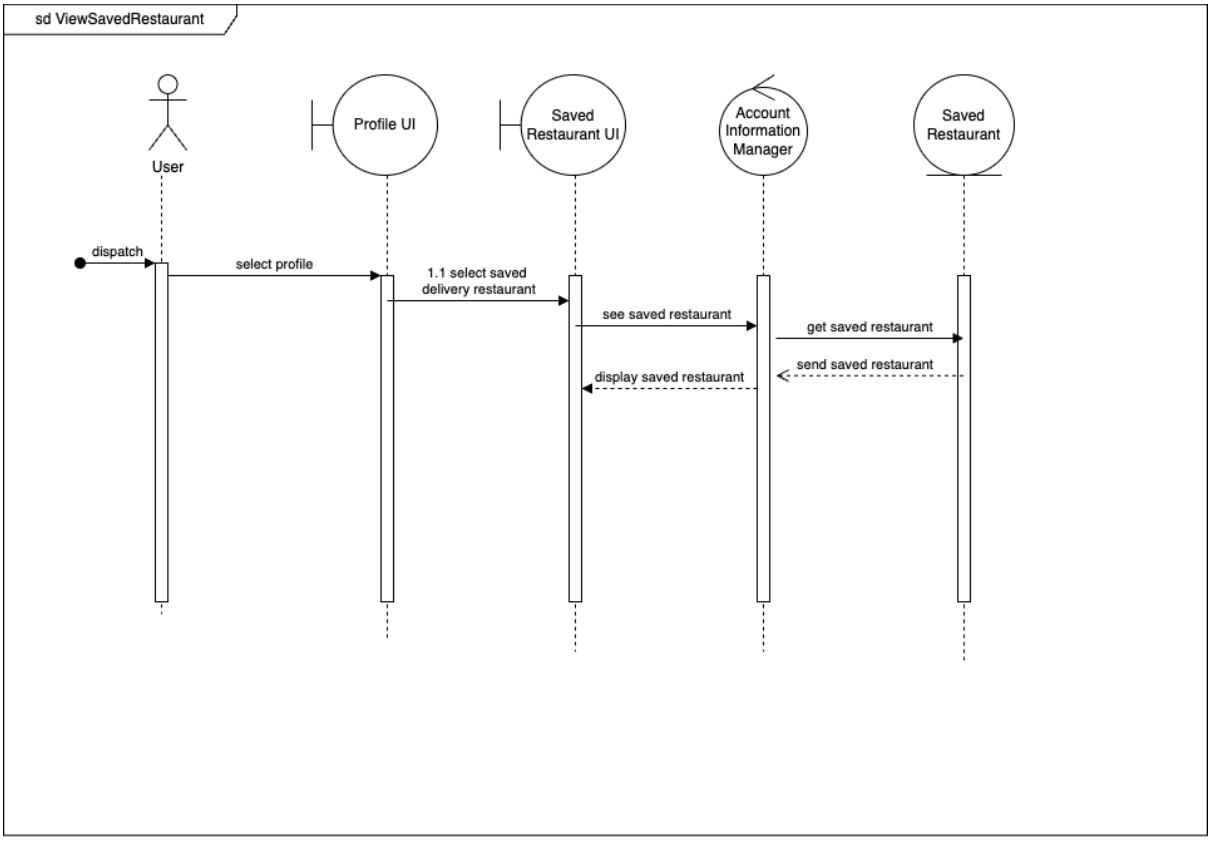

sd UploadRecipe
User
Upload Recipe
Recipe Manager
Recipe
select upload
fill details
store(recipe)
send(uploaded)
send(uploaded)
send(uploaded)


sd ViewEditDeleteOwnRecipe
User
Manage Recipes UI
Uploaded Recipe UI
Recipes
alt
[ownRecipesExist==true]
views their recipes
view details
alt
detele==true
delete
edit==true
edit
stores(updatedRecipe)

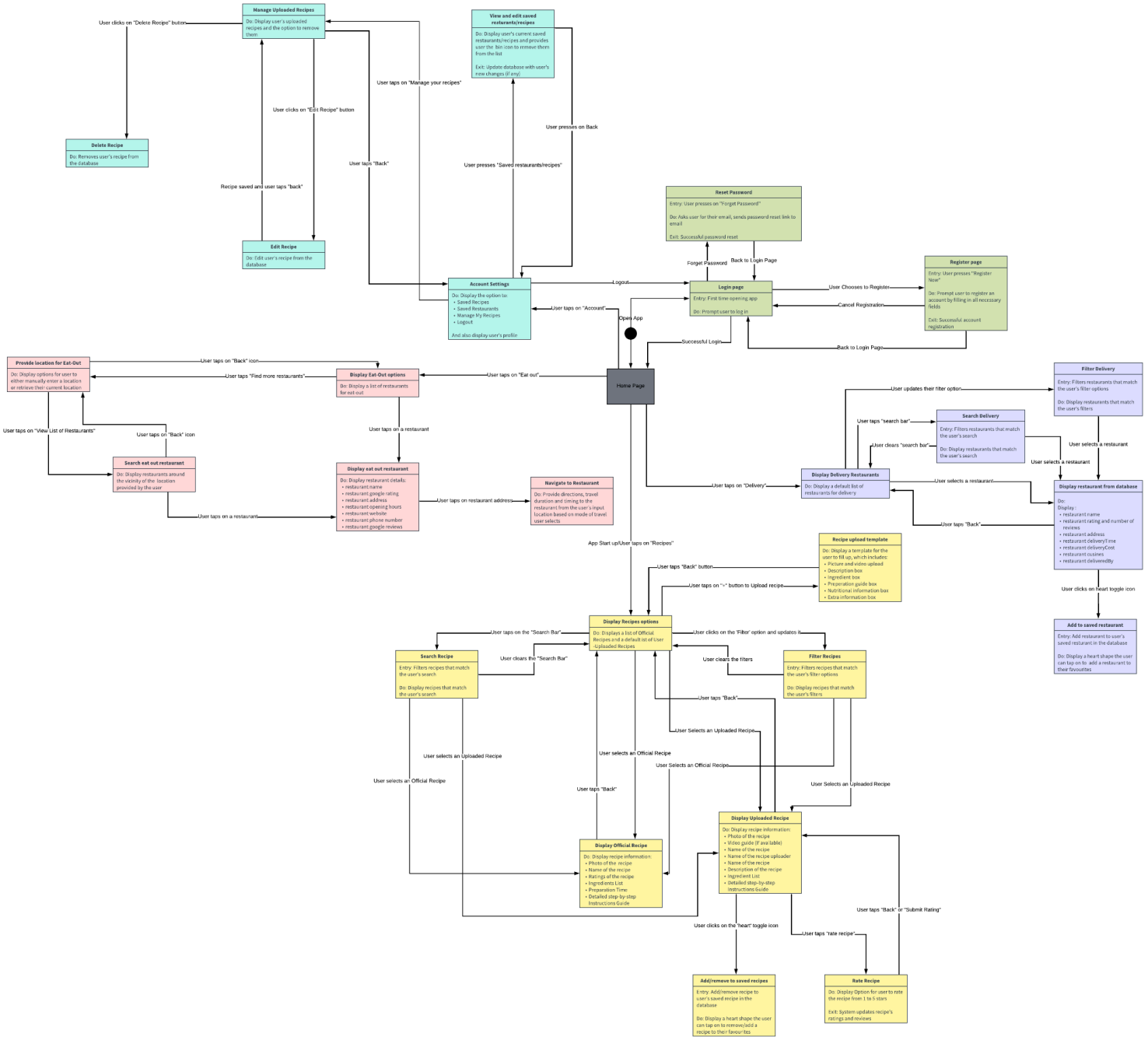

sd SearchRestaurantEatOut
User
Search for RestaurantsUI
Restaurant Manager
Geolocation API
Google Places API
taps on Eat-out option
get restaurant info
returns restaurants info
asks user for location preference
choose option
alt
option(1):
give access to their location
sends user's location
option(2):
enter their location manually
location
returns nearby restaurants
Option():
1 location
2 search bar
3 saved locations


sd SearchRestaurantDelivery
User
Searchby Name Restaurants UI
Restaurant Searchby Name Manager
Restaurant Searchby Ratng Manager
Restaurant Database
Searchby Rating Restaurants UI
taps on Delivery option
alt
Searchby Rating
Searchby Rating
get restaurant info
returns restaurants info
filters restaurants
sends list of filtered restaurants
chooses restaurant
get restaurant info
returns restaurants info
return
Searchby Name
get restaurant info
returns restaurants info
filters restaurants
sends list of filtered restaurants
chooses restaurant
get restaurant info
returns restaurants info

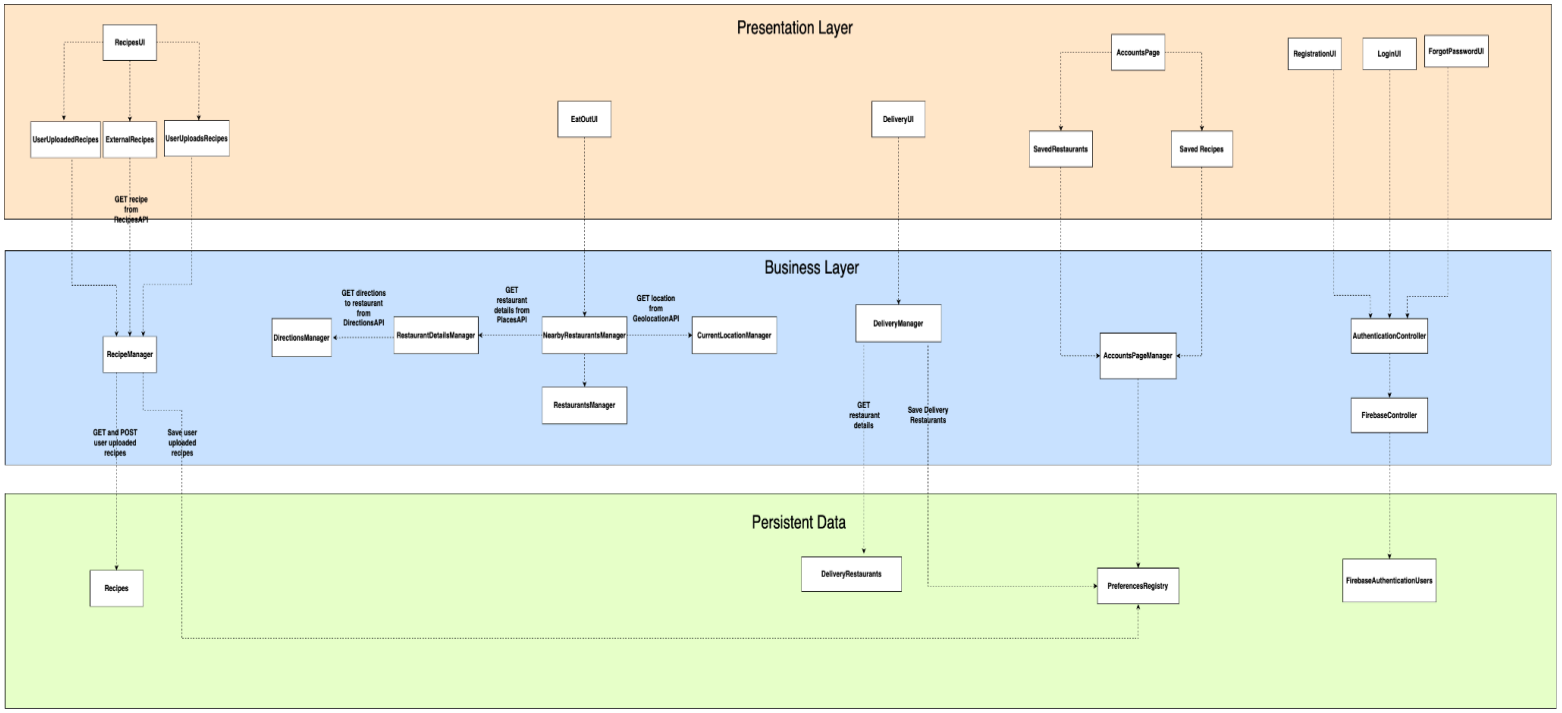

sd SaveRestaurant
User
List of Restaurants UI
Restaurant Manager
Saved Restaurant
see restaurant details
save(restaurant)
save(restaurant)
update saved status
update recipe saved status
display recipe saved status


sd ViewSavedRestaurant
User
Profile UI
Saved Restaurant UI
Account Information Manager
Saved Restaurant
dispatch
select profile
1.1 select saved delivery restaurant
see saved restaurant
get saved restaurant
send saved restaurant
display saved restaurant

Dialog Map


Manage Uploaded Recipes
Do: Display user's uploaded recipes and the option to remove them
User clicks on "Delete Recipe" button
Delete Recipe
Do: Removes user's recipe from the database
User clicks on "Edit Recipe" button
Recipe saved and user taps "back"
Edit Recipe
Do: Edit user's recipe from the database
User taps on "Manage your recipes"
User taps "Back"
View and edit saved restuarants/recipes
Do: Display user's current saved restaurants/recipes and provides user the bin icon to remove them from the list
Exit: Update database with user's new changes (if any)
User presses on Back
User presses "Saved restaurants/recipes"
Reset Password
Entry: User presses on "Forget Password"
Do: Asks user for their email, sends password reset link to email
Exit: Successful password reset
Forget Password
Back to Login Page
Account Settings
Do: Display the option to:
• Saved Recipes
• Saved Restaurants
• Manage My Recipes
• Logout
And also display user's profile
Logout
User taps on "Account"
Login page
Entry: First time opening app
Do: Prompt user to log in
User Chooses to Register
Cancel Registration
Register page
Entry: User presses "Register Now"
Do: Prompt user to register an account by filling in all necessary fields
Exit: Successful account registration
Back to Login Page
Opens App
Successful Login
Provide location for Eat-Out
Do: Display options for user to either manually enter a location or retrieve their current location
User taps on "Back" icon
User taps "Find more restaurants"
Display Eat-Out options
Do: Display a list of restaurants for eat-out
User taps on "Eat out"
Home Page
User taps on "View List of Restaurants"
User taps on "Back" icon
Search eat out restaurant
Do: Display restaurants around the vicinity of the location provided by the user
User taps on a restaurant
Display eat out restaurant
Do: Display restaurant details:
• restaurant name
• restaurant google rating
• restaurant address
• restaurant opening hours
• restaurant website
• restaurant phone number
• restaurant google reviews
User taps on restaurant address
Navigate to Restaurant
Do: Provide directions, travel duration and timing to the restaurant from the user's input location based on mode of travel user selects
Filter Delivery
Entry: Filters restaurants that match the user's filter options
Do: Display restaurants that match the user's filters
User updates their filter option
User taps "search bar"
Search Delivery
Entry: Filters restaurants that match the user's search
Do: Display restaurants that match the user's search
User clears "search bar"
User selects a restaurant
User selects a restaurant
Display Delivery Restaurants
Do: Display a default list of restaurants for delivery
User taps on "Delivery"
User selects a restaurant
Display restaurant from database
Do:
Display :
• restaurant name
• restaurant rating and number of reviews
• restaurant address
• restaurant deliveryTime
• restaurant deliveryCost
• restaurant cuisines
• restaurant deliveredBy
User taps "Back"
User clicks on heart toggle icon
Add to saved restaurant
Entry: Add restaurant to user's saved restuarant in the database
Do: Display a heart shape the user can tap on to add a restaurant to their favourites
App Start up/User taps on "Recipes"
Recipe upload template
Do: Display a template for the user to fill up, which includes:
• Picture and video upload
• Description box
• Ingredient box
• Preperation guide box
• Nutritional information box
• Extra information box
User taps "Back" button
User taps on "+" button to Upload recipe
Display Recipes options
Do: Displays a list of Official Recipes and a default list of User Uploaded Recipes
User taps on the "Search Bar"
User clicks on the 'Filter' option and updates it
Search Recipe
Entry: Filters recipes that match the user's search
Do: Display recipes that match the user's search
User clears the "Search Bar"
User clears the filters
Filter Recipes
Entry: Filters recipes that match the user's filter options
Do: Display recipes that match the user's filters
User taps "Back"
User Selects an Uploaded Recipe
User selects an Uploaded Recipe
User selects an Official Recipe
User selects an Official Recipe
User Selects an Official Recipe
User taps "Back"
User Selects an Uploaded Recipe
Display Uploaded Recipe
Do: Display recipe information:
• Photo of the recipe
• Video guide (if available)
• Name of the recipe
• Name of the recipe uploader
• Name of the recipe
• Description of the recipe
• Ingredient List
• Detailed step-by-step Instructions Guide
Diaplay Official Recipe
Do: Display recipe information:
• Photo of the recipe
• Name of the recipe
• Ratings of the recipe
• Ingredients List
• Preparation Time
• Detailed step-by-step Instructions Guide
User taps "Back" or "Submit Rating"
User clicks on the 'heart' toggle icon
User taps "rate recipe"
Add/remove to saved recipes
Entry: Add/remove recipe to user's saved recipe in the database
Do: Display a heart shape the user can tap on to remove/add a recipe to their favourites
Rate Recipe
Do: Display Option for user to rate the recipe from 1 to 5 stars
Exit: System updates recipe's ratings and reviews

System Architecture and Design



For our system design, we adopted this three-layered architecture to streamline operations and enhance maintainability.

Presentation Layer: This topmost layer handles all user interactions via interfaces such as DeliveryUI, EatOutUI, and RecipesUI. These interfaces allow users to perform actions like searching for and uploading recipes. Authentication processes are managed through RegistrationUI, LoginUI, and ForgotPasswordUI.

Business Layer: The Business Layer bridges the Presentation and Data Layers. It processes user requests and coordinates with external APIs. Managers like RecipeManager, DirectionsManager, and DeliveryManager facilitate these interactions. For instance, NearbyRestaurantsManager employs the GeolocationAPI to identify restaurants near the user.

Data Layer: The foundational layer of our architecture, the Data Layer, is tasked with data management. For example, it integrates databases and registries such as Recipes and FirebaseAuthenticationUsers, which are pivotal for storing recipe information and managing user accounts.

This three-layered architecture promotes a clear separation of concerns, where each layer has a specific role and communicates with the others through well-defined interfaces. This organisation not only simplifies system maintenance and testing but also enhances the scalability and hence adaptability of our application.

Design Practices

1. Extensibility : By utilising modular component systems, our design ensures that each feature can be extended and customised without significant rework. For example, if we decide to introduce a new type of recipe filter, we can add a component to our existing filter options without altering the underlying architecture.

2. Single Responsibility Principle (SRP): We've adhered to SRP as each class and widget is assigned the responsibility for a single part of the application's functionality. We have separate classes for UI implementations of each page and API calls etc. This makes the code easier to understand, debug, and maintain since changes in one part of the system have minimal impact on others.

3. Use of Asynchronous Programming: Using asynchronous programming has been crucial for maintaining a responsive user interface. When our app interacts with APIs or databases, it does so without blocking the user interface, thanks to async functions and Future objects. This way, users can enjoy a seamless experience even during complex operations like fetching large numbers of recipes or waiting for a list of restaurants to load.

Software Engineering Practices

1. Agile Development: Our team has embraced agile methodology, which has allowed us to adapt our plans and features based on iterative testing. Some of the experiences we had were:
    - Due to frequent changes in our designs, it made the development of our project really unstable. It required us to adjust our timelines many times so as to not exceed it.
    - While designing the eat-out section, we decided to add an additional feature, which is providing the user navigation directions to the restaurant, to enhance user experience.

2. Modular Architecture: By structuring the app with a modular design philosophy we could develop, test, and debug in isolation, enhancing the robustness of the app. Each module, like user authentication or restaurant searching, functions independently, reducing dependencies and potential bugs during integration.

Testing

We have used both black and white box testing techniques to test Trinity's functionalities rigorously.
We employed black box testing techniques for our features that involve exceptions, to ensure that each feature operated as expected from the end-user perspective and at the same time validated against our defined functional requirement.
Simultaneously, we adopted white box testing methods to scrutinise the certain control flows within the app such as login/register and crucial aspects of each of our core functionalities. This allowed us to examine the logical paths, verify the integrity of any conditions and branches and also ensure error handling was robust, enhancing the reliability and performance of our application's functions.

Black Box Testing

1. Login

1.1 Login Using Email And Password

Test Input		**Test output**	
Email	**Password**	**Expected Output**	**Actual Output**
Valid/Correct email (eg. ananya@gmail.com)	Correct password (eg. Trinity1@)	Get logged in successfully and redirect to Recipes Main page	Successfully logged in and redirected to Recipes Main page
Incorrect email (eg. ananya327@gmail.com)	Correct password (eg. Trinity1@)	Display error message: "Error: The supplied auth credential is incorrect, malformed or has expired"	Error message displayed: "Error: The supplied auth credential is incorrect, malformed or has expired"
Invalid email (eg. ananya)	Correct password (eg. Trinity1@)	Display error message: "The email address is badly formatted"	Error message displayed: "The email address is badly formatted"
Valid/Correct email (eg. ananya@gmail.com)	Incorrect password (eg. trinity)	Display error message: "Error: The supplied auth credential is incorrect, malformed or has expired"	Error message displayed: "Error: The supplied auth credential is incorrect, malformed or has expired"

1.2 Forgot Password

Test Input	Expected Output	Actual Output
Valid/Correct email (eg. ananya@gmail.com)	Display message: “Password reset email sent” and return to Login Main Page	Message displayed: “Password reset email sent” and return to Login Main Page
Incorrect email (eg. ananya123@gmail.com)	Password reset email cannot be sent	Password reset email cannot be sent
Invalid email (eg. ananya)	Display error message: “The email address is badly formatted” and return to Login Main Page	Error message displayed: “The email address is badly formatted” and returned to Login Main Page

2. Register For A New Account

2.1 Email And Password/Confirm Password Combinations

Test Input		Test Output	
Email	**Password/Confirm Password**	**Expected Output**	**Actual Output**
Valid/Available Email (eg. ananya123@gmail.com)	Trinity1@	Email Verification link sent to email address	Email Verification link sent to email address
	abcd	Password field displays: “Minimum 8 characters required”	Password field displayed: “Minimum 8 characters required”
	abcdefghij1@	Password field displays: “Maximum 12 characters accepted”	Password field displayed: “Maximum 12 characters accepted”
	password	Password field displayed: “Should include 1) a digit 2) a lowercase character 3) an uppercase character 4) a special	Password field displayed: “Should include 1) a digit 2) a lowercase character 3) an uppercase character 4) a special

		character”	character”
	password1	Password field displayed: “Should include 1) a digit 2) a lowercase character 3) an uppercase character 4) a special character”	Password field displayed: “Should include 1) a digit 2) a lowercase character 3) an uppercase character 4) a special character”
	password@	Password field displayed: “Should include 1) a digit 2) a lowercase character 3) an uppercase character 4) a special character”	Password field displayed: “Should include 1) a digit 2) a lowercase character 3) an uppercase character 4) a special character”
	Password	Password field displayed: “Should include 1) a digit 2) a lowercase character 3) an uppercase character 4) a special character”	Password field displayed: “Should include 1) a digit 2) a lowercase character 3) an uppercase character 4) a special character”
	Password1	Password field displayed: “Should include 1) a digit 2) a lowercase character 3) an uppercase character 4) a special character”	Password field displayed: “Should include 1) a digit 2) a lowercase character 3) an uppercase character 4) a special character”
	Password@	Password field displayed: “Should include 1) a digit 2) a lowercase character 3) an uppercase character	Password field displayed: “Should include 1) a digit 2) a lowercase character 3) an uppercase character

		4) a special character”	4) a special character”
Invalid email (eg. ananya)	Password that meets all requirements (eg. Trinity1@)	Email field displays: “Email is invalid”	Email field displayed: “Email is invalid”
Valid/Unavailable email (eg. ananya@gmail.com	Password that meets all requirements (eg. Trinity1@)	Displays error message: “Error: The email is already in use by a different account”	Error message displayed: “Error: The email is already in use by a different account”

2.3 Password And Confirm Password Combinations (when email is valid/available)

Test Input		**Test Output**	
Password	**Confirm Password**	**Expected Output**	**Actual Output**
Password1@	password1	Display error message: “Passwords don’t match!”	Error message displayed: “Passwords don’t match!”
Password1@	Password1@	Email Verification link sent to email address and redirect to Login Main Page	Email Verification link sent to email address and redirected to Login Main Page

3. Recipe

3.1 Searching For Recipes And Applying Filters

Test Input		**Test Output**	
Search by text input	**Filters**	**Expected Output**	**Actual Output**
Enter a dish name (eg. pasta)	No filters	Dishes with the entered name	Three pasta dishes - chicken alfredo pasta, shrimp scampi pasta, pesto pasta)
No searching	Filter by Medium Difficulty	Dishes with medium difficulty	Dishes with medium difficulty

Enter a dish name (eg. pasta)	Filter by Medium Difficulty	Dishes with the entered name and medium difficulty	chicken alfredo pasta, shrimp scampi pasta which have medium difficulty

3.2 Uploading your own recipe

Test Input					**Test Output**	
Recipe name	**Ingredients list**	**Preparation steps**	**Cuisine type, difficulty level, Meal type**	**Other fields**	**Expected Output**	**Actual Output**
Enter Recipe Name	Enter Ingredients list	Enter preparation steps	Choose these options from drop-down menu	Some entered, some kept empty	Should upload the recipe successfully and display it in the User recipes list as well as manage my uploaded recipes in Accounts Section	Recipe uploaded successfully and displayed in the User recipes list as well as manage my uploaded recipes in Accounts Section
Leave empty	Enter Ingredients list	Enter preparation steps	Choose these options from drop-down menu	Some entered, some kept empty	Recipe name 'field is required' warning	Recipe name field is required warning
Enter Recipe Name	Leave empty	Enter preparation steps	Choose these options from drop-down menu	Some entered, some kept empty	Ingredients list 'field is required' warning	Ingredients list 'field is required' warning
Enter Recipe Name	Enter Ingredients list	Leave empty	Choose these options from drop-down menu	Some entered, some kept empty	Preparation steps 'field is required' warning	Preparation steps 'field is required' warning
Enter Recipe Name	Enter Ingredient	Enter preparatio	Don't choose	Some entered,	'Difficulty level' cannot be empty	'Difficulty level' cannot be empty

	s list	n steps	some options (eg. difficulty level)	some kept empty	warning	warning

3.3 Saving a user uploaded recipe

Test Input	**Expected Output**	**Actual Output**
Clicking on the heart to save the recipe which has not already been saved	Added to favourites	Added to favourites, and can view on accounts section->saved recipes.
Click on the heart to save the recipe which has already been saved	Recipe already saved warning	Recipe already saved warning

4. Eat-Out

4.1 Viewing List of Restaurants

Test Input		Expected Output	Actual Output
Valid location provided	Click on 'View List of Restaurants' button	List of restaurants within 2km radius of user's given location are displayed	List of restaurants within 2km radius of user's given location are displayed
Valid location not provided	Click on 'View List of Restaurants' button	Empty page with no results is displayed	Empty page with no results is displayed

4.2 Viewing Restaurant Navigation Routes

Test Input		**Expected Output**	**Actual Output**
Initial Location	Mode of Travel	Redirected to the respective page to show navigation directions from initial	Redirected to the respective page to show navigation directions from initial
Enter and choose initial location manually:	Click on preferred mode of travel		

“Boon Lay Interchange, Singapore”		chosen location to the restaurant	chosen location to the restaurant
Initial location not selected	Click on preferred mode of travel	Empty page with no directions is displayed	Empty page with no directions is displayed

5. Delivery

5.1 Search for restaurants manually by typing in the name/cuisine

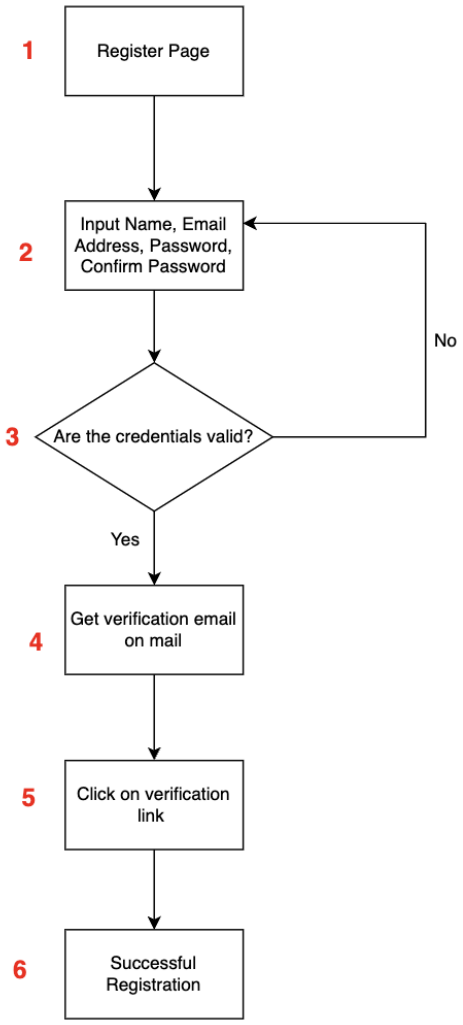
Test Input	**Expected Output**	**Actual Output**
Enter in the search bar (eg. green)	Display Restaurants starting with the search text	Display Restaurants starting with the search text (eg. green dot, green signature, etc.) along with information about their location, ratings and cuisines served.
Enter a restaurant name/cuisine which doesn’t exist	Should not show any restaurant	Doesn’t show any restaurant

White Box Testing (Basis Path Testing)

For selected control flows in application

CC = decisionpoints + 1 (for binary decision points)
= edges-nodes + 2

1.Registering for a new account

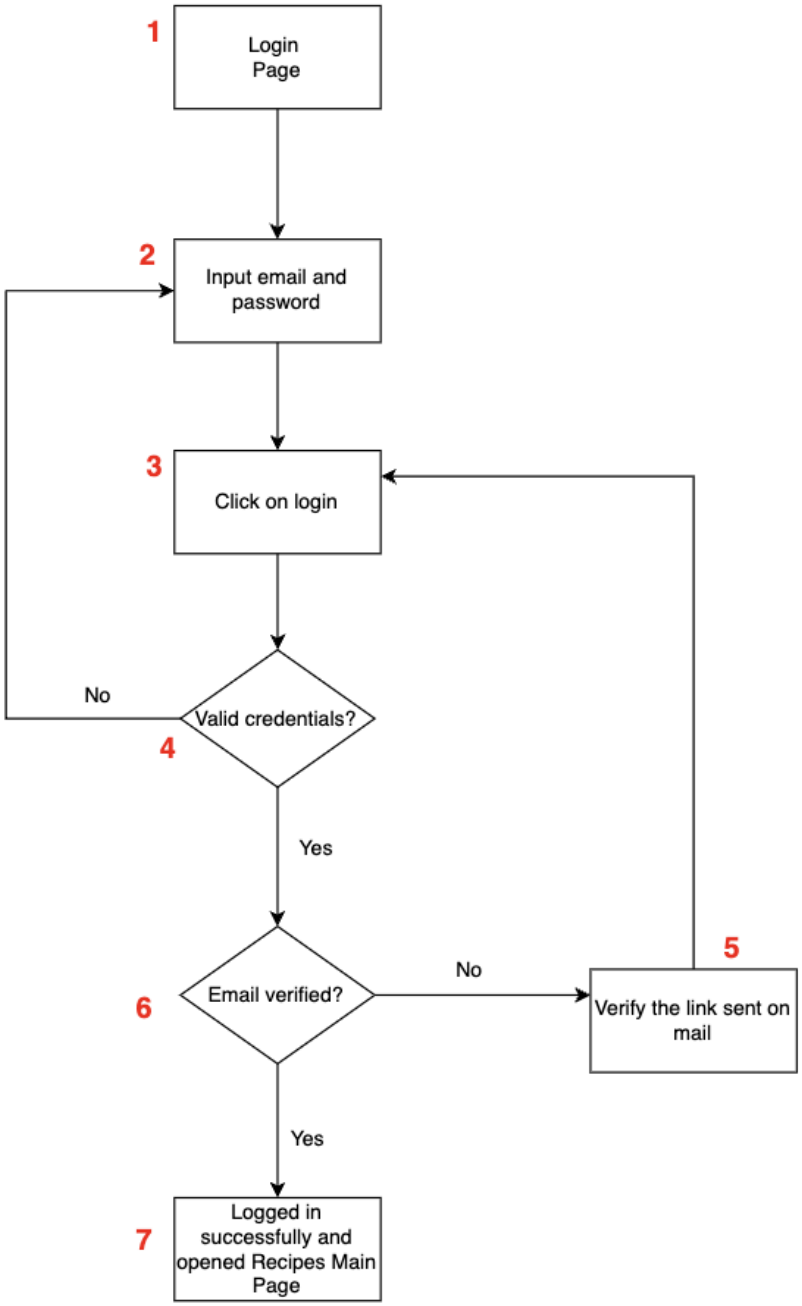


CC = |decisionpoints| +1 = 2
Baseline Basis path = 1,2,3,4,5,6
Set of Basis Path:

1) 1,2,3,4,5,6
2) 1,2,3,2,3,4,5,6

	Test Input				**Outputs**	
Basis Path	**Name**	**Email**	**Password**	**Confirm Password**	**Expected Output**	**Actual Output**
1,2,3,4,5,6	Ananya	Ananya @gmail.com	Ananya2@	Ananya2 @	Registered Successfully and navi	Verification link sent
1,2,3,2,3,4,5,6	Ananya	ananya@gmail.com	Ananya2@	annnanya2@	Password doesn't match, then enter valid credentials and register successfully	Password doesn't match, then enter valid credentials and register successfully

2. Login

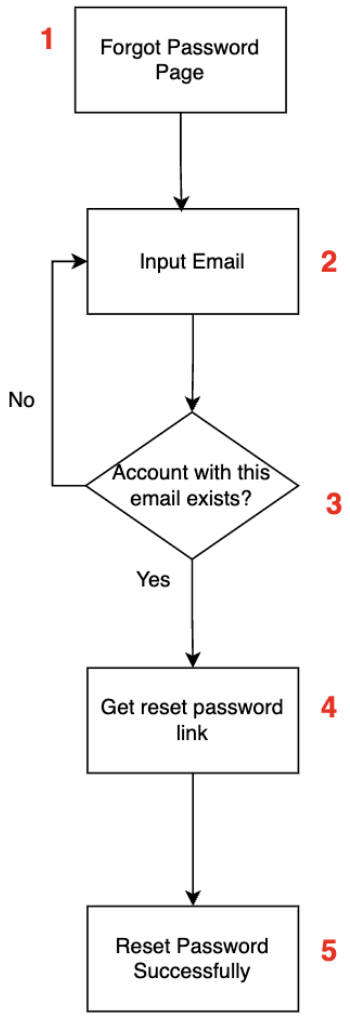


CC=2+1=3
Baseline Basis path = 1,2,3,4,6,7
Set of Basis Path:

1) 1,2,3,4,6,7
2) 1,2,3,4,2,3,4,6,7
3) 1,2,3,4,6,5,3,4,6,7

	Test Input			Outputs	
Basis Path	**Email**	**Password**	**Email verified**	**Expected Output**	**Actual Output**
1,2,3,4,6,7	Correct email (Ananya @gmail.com)	Correct Password (Ananya2@)	Yes	Logged in successfully and navigated to recipes main page	Logged in successfully and navigated to recipes main page
1,2,3,4,2,3 ,4,6,7	Correct email (Ananya @gmail.com)	Incorrect Password (Ananya2@)	Yes	1)Error due to wrong password 2)Logged in successfully after inputting valid credentials	1)Error due to wrong password 2)Logged in successfully after inputting valid credentials
1,2,3,4,6,5 ,3,4,6,7	Correct email (Ananya @gmail.com)	Correct Password (Ananya2@)	No	1)Ask to verify email 2)Login successfully after verification	1)Ask to verify email 2)Login successfully after verification

3. Forgot Password



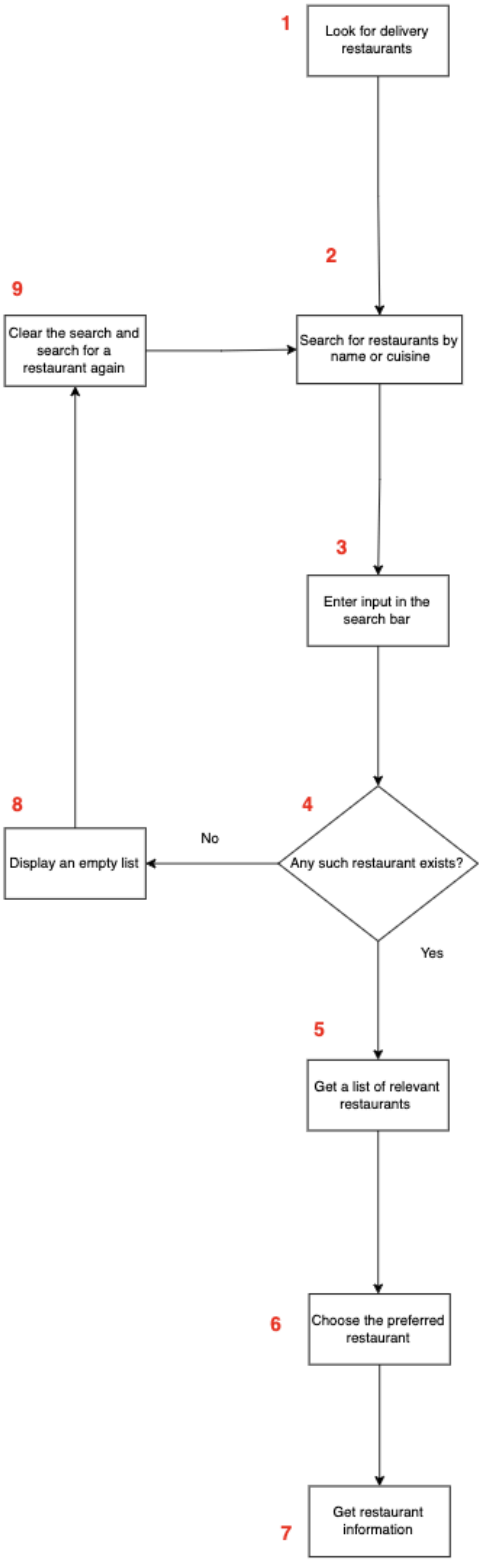
CC=1+1=2
Baseline Basis path = 1,2,3,4,6,7
Set of Basis Path:

1) 1,2,3,4,5
2) 1,2,3,2,3,4,5

	Test Input	**Outputs**	
Basis Path	**Email**	**Expected Output**	**Actual Output**
1,2,3,4,5	Account exists(Ananya @gmail.com)	Reset password link sent and password changed successfully	Reset password link sent and password changed successfully
1,2,3,2,3,4 ,5	Account doesn't exists(Ananya12	1)Not able to send reset link 2)Input correct email so	1)Not able to send reset link 2)Input correct email so Reset

	@gmail.com)	Reset password link sent and password changed successfully	password link sent and password changed successfully

4. Searching for restaurants for delivery by their name or cuisine they serve

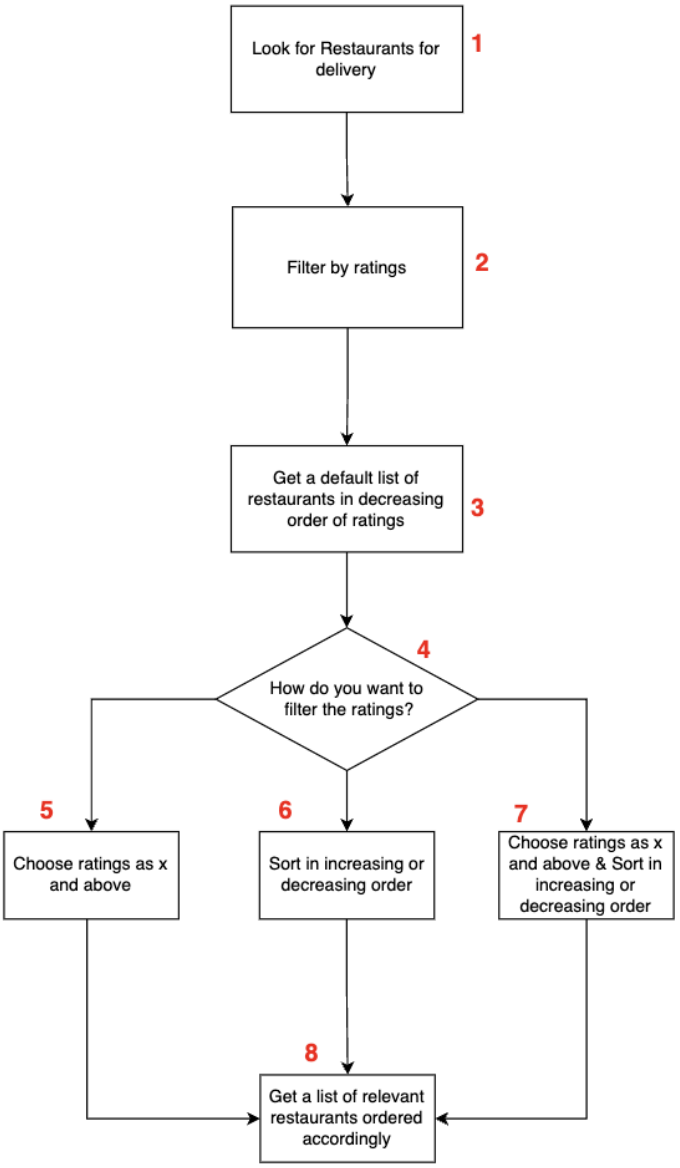


CC= 1+1=2
Baseline basis path = 1,2,3,4,5,6,7
Set of basis paths:

1) 1,2,3,4,5,6,7
2) 1,2,3,4,8,9,2,3,4,5,6,7

	Test Input	**Outputs**	
Basis Path	**Search input**	**Expected Output**	**Actual Output**
1,2,3,4,5,6,7	Restaurant exists(eg. Thai)	Gives a list of relevant restaurants (eg. tenoy thai kitchen, ting tong thai etc.)	Reset password link sent and password changed successfully
1,2,3,4,8,9,2,3,4,5,6,7	Restaurant doesn't exist(eg. gibberish)	Displays an empty list	Displays an empty list

5. Searching for restaurants for delivery by their ratings



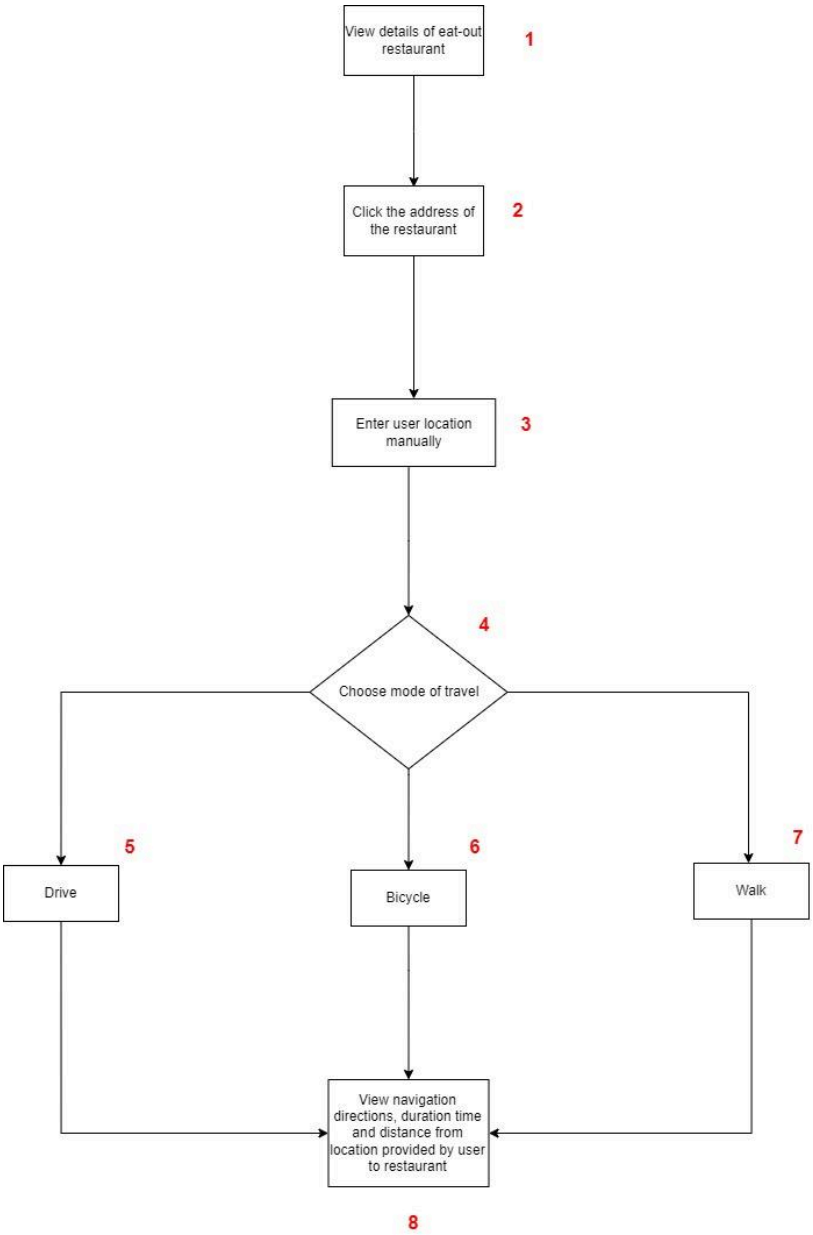
CC= edges-nodes+2=9-8+2=3
Baseline basis path = 1,2,3,4,7,8
Set of basis paths:

1) 1,2,3,4,7,8
2) 1,2,3,4,6,8
3) 1,2,3,4,5,8

	Test Input	Outputs

Basis Path	Rating	Expected Output	Actual Output
1,2,3,4,7,8	Choose 3 and above & switch to increasing order	Gives a list of restaurants ordered in increasing order of rating from 3 to 5.	Gives a list of restaurants ordered in increasing order of rating from 3 to 5.
1,2,3,4,6,8	Switch to increasing order	Gives a list of restaurants ordered in increasing order of rating from 1 to 5.	Gives a list of restaurants ordered in increasing order of rating from 1 to 5.
1,2,3,4,5,8	Choose 3 and above	Gives a list of restaurants or ratings 3 and above and ordered in decreasing if default or ordered in increasing if switched to increasing order.	Gives a list of restaurants or ratings 3 and above and ordered in decreasing if default or ordered in increasing if switched to increasing order.

6. View navigation directions to eat-out restaurant

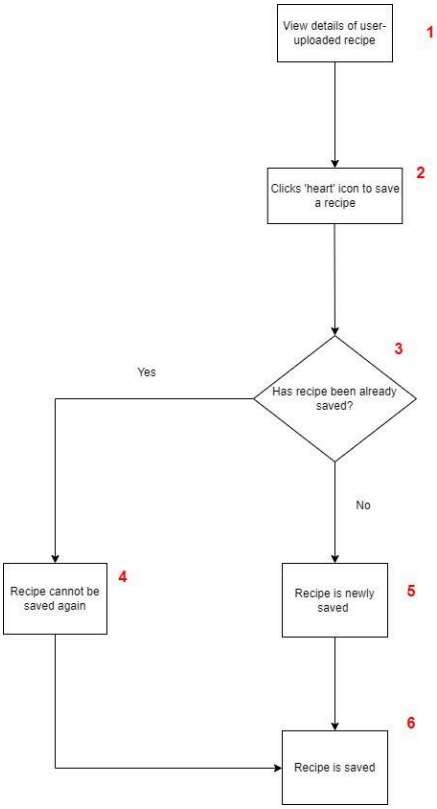


CC = edges - nodes + 2 = 9 - 7 + 2 = 4
Baseline basis path = 1, 2, 3, 4, 5, 8
Set of basis paths:

1) 1, 2, 3, <u>4, 5</u>, 8
2) 1, 2, 3, <u>4, 6</u>, 8
3) 1, 2, 3, <u>4, 7</u>, 8

	Test Input	Outputs	
Basis Path	**Mode of travel**	**Expected Output**	**Actual Output**
1, 2, 3, 4, 5, 8	Drive	Provides driving directions, duration time and distance from location provided by user to restaurant	Provides driving directions, duration time and distance from location provided by user to restaurant
1, 2, 3, 4, 6, 8	Bicycle	Provides cycling directions, duration time and distance from location provided by user to restaurant	Provides cycling directions, duration time and distance from location provided by user to restaurant
1, 2, 3, 4, 7, 8	Walk	Provides walking directions, duration time and distance from location provided by user to restaurant	Provides walking directions, duration time and distance from location provided by user to restaurant

7. Saving a user-uploaded recipe



CC = edges - nodes + 2 = 6 - 6 + 2 = 2
Baseline basis path = 1, 2, 3, 5, 6
Set of basis paths:

1) 1, 2, 3, 5, 6
2) 1, 2, 3, 4, 6

	Test Input	**Outputs**	
Basis Path	**Saved Recipe**	**Expected Output**	**Actual Output**
1, 2, 3, 4, 6	Recipe is already saved	Display error message: “Recipe Already Saved”	Display error message: “Recipe Already Saved”
1, 2, 3, 5, 6	Recipe is not saved	Display message: “Added to favourites” and recipe is saved	Display message: “Added to favourites” and recipe is saved

Appendix

1. All the diagrams in the report are uploaded separately as pdf files for clearer view.

2. To take a look at our application, watch our demo video which has been uploaded on youtube here.
   (Youtube Link: https://youtu.be/m6YwMhsjWZ8?si=lOb-rljwLFM67IM9)

3. We have also provided an .apk file which you can open on your emulator or your android device.